Noname manuscript No.
(will be inserted by the editor)

Partial Order Reduction for Abstraction-based Verification of Concurrent Software

Csanád Telbisz · **Levente Bajczi** ·
Dániel Szekeres · **András Vörös**

the date of receipt and acceptance should be inserted later

Abstract Formal verification of concurrent software faces two major challenges: the state space explosion problem from data complexity, and the combinatorial explosion of thread interleavings. Abstraction and partial order reduction (POR) are well-established techniques for addressing these challenges individually, yet their combination has received comparatively little attention. In this paper, we investigate the principled integration of POR with abstract state space exploration, without imposing restrictions on the underlying POR algorithm or abstract domain. We use an abstraction-aware dependency relation for deciding the commutativity of actions from different threads that leverages the precision of the abstraction, considering two actions independent when the abstraction tracks no information about their shared variables. We reformulate the necessary and sufficient conditions for the correctness of any POR algorithm applied in an abstract state space building on this abstraction-aware concept of dependency. We also present a concrete POR algorithm as a case study of the general approach and prove its soundness. We implement multiple partial order reduction algorithms with the proposed abstraction-aware extension to demonstrate the applicability of the general approach and evaluate the method in two verification frameworks, using different abstract domains. The experimental results reveal the increased reduction potential of the proposed approach in abstract state spaces. The practical evaluation also confirms the theoretical possibility of exponential reduction in certain benchmarks enabled by the abstraction-aware extension.

Keywords model checking · concurrency · partial order reduction · abstraction

*Funding.*This research was partially funded by the EKÖP-25-3 New National Excellence Program under project number EKÖP-25-3-BME-22, and the Doctoral Excellence Fellowship Programme under project number DKÖP-23-1-BME-5; funded by the National Research Development and Innovation Fund of the Ministry of Culture and Innovation of Hungary.

Corresponding author: Csanád Telbisz, telbisz@mit.bme.hu

Department of Artificial Intelligence and Systems Engineering
Budapest University of Technology and Economics
Budapest, Hungary

1 Introduction

Software verification suffers from the state space explosion problem, as the size of the state space of a program often grows exponentially with the number of variables and the size of their domains [18]. An efficient approach to handle this problem is abstraction [17,26]. Rapid development in technology led to the increasing popularity of multi-core processors and multi-threaded programs in industrial systems. Concurrency introduces a new level of complexity to software verification due to the great number of possible thread interleavings. Partial order reduction (POR) is a well-known technique for handling this complexity of concurrency in model checking [1, 36].

The core concept of POR is to identify equivalent executions [24]. Then, it is enough to check a single representative from each equivalence class. Identifying equivalent interleavings is based on the interaction of threads: dependency is defined between the interacting program operations. Traditionally, a syntactic over-approximation is used as a method of calculating dependency for partial order reduction: two actions are independent if they do not use common shared variables (and the two actions belong to different processes) [24]. For example, let us have a state in the (concrete) state space of the program with two enabled actions from different threads: `x:=1` and `y:=1`. No matter in what order we explore the two actions (`x:=1`, `y:=1` or `y:=1`, `x:=1`), we will reach the same state since the actions operate on different variables. So we can skip the exploration of one of the two paths.

While partial order reduction (POR) is an effective technique for handling concurrency, abstraction is an efficient approach to handling data in model checking. This motivates the combination of the two methods in a verification algorithm for multi-threaded programs. There are numerous works in the literature that study the two techniques in isolation, but few of them combine the two approaches [20, 25]. It is an even smaller subset that actually leverages the benefits of applying abstraction and partial order reduction together. Those methods are either specific to a POR or an abstraction algorithm [16, 31, 38, 44], or pose limitations that not all partial order reduction algorithms meet [21, 29]. This motivates our study, which investigates the combination of partial order reduction and abstract state space exploration without imposing limitations on the partial order reduction algorithm. The necessary conditions for the correctness of any POR algorithm (without abstraction) have been formulated in the literature [3]. We identify the key differences of using partial order reduction in an abstract state space and reformulate the necessary condition accordingly.

When it comes to combining POR with abstraction, we face the problem that traditional approaches may calculate invalid dependency relations: it is not trivial to apply POR in an abstract state space where the values of variables are not tracked explicitly [16,37]. Example 1.1 shows a situation where syntactically independent actions are not commutative in an abstract state space.

Example 1.1 Assume that we have an abstraction where we only track the predicates `x > y` and `x > y + 1` about our variables. Figure 1.1 demonstrates that two actions that are independent in the traditional sense (no shared variable) may not be commutative in the abstract state space. In reality, the actions `x++` and `y++` are dependent since they are not commutative in the abstract state space due to the difference in the labeling of the reached abstract states. Later, Figure 3.2 shows a case where actions with different variables can even disable each other.

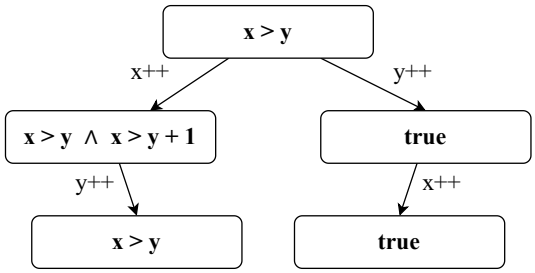


Fig. 1.1: Syntactically independent actions that are not commutative in the abstract state space

Even though the syntactic over-approximation of dependency is not a valid dependency relation in the abstract state space, we show that using it for partial order reduction will always find an error state in the abstract state space if an error is reachable in the concrete state space. Furthermore, we restrict the dependency relation to consider fewer actions dependent in an abstract state space. Intuitively, if the source of dependency between two actions is ignored due to abstraction, it is needless to consider these actions dependent. That is, two actions using the same shared variable are only considered dependent if we track any information about any of their shared variables in the abstraction. As a basic example, take the actions `x:=0` and `x:=1`. Traditional methods consider these actions dependent since they both write `x`. However, our approach allows these actions to be independent when we do not track any information about x in the abstraction.

While relaxing the commutativity condition is not a novel idea [21], we propose a way of integrating this concept in a model checking algorithm using partial order reduction without putting any limitation on the actual POR algorithm. That is, we formulate conditions for the correctness of an arbitrary POR algorithm using the abstraction-aware dependency relation in an abstract state space. We also present a concrete verification algorithm as a case study of our general approach implementing the proposed combination of POR and abstraction. The novelty of our work lies in the general formulation of POR used during abstract state space exploration.

To further motivate our approach, it is possible to achieve exponential gains in terms of the number of explored interleavings by using our abstraction-aware algorithm for POR. Consider the example from Figure 1.2 with $2N+1$ processes. This is indeed a challenging task for verifiers: the program is the pseudo-code of a C program used in the international Competition on Software Verification (SV-COMP) [14], and even state-of-the-art tools (such as Ultimate [30] and Dartagnan [32]) struggle to correctly solve the problem within a reasonable time as N grows[1]. The safety of the program can be proven with abstraction by only tracking the predicates `z` mod `2 = 0` and `x = 0` about our variables. As `z` mod `2 = 0` is an invariant of the loop of p_0, x will get the value 0 which satisfies the assertion. To prove this, traditional methods would explore all interleavings of instructions using `y` since these actions would be considered dependent due to `y`: processes $p_1 - p_{2N}$ have $2N!$ interleavings (potentially resulting in different values of `y`) not considering the loop of p_0; together there are even more possible

[1] See results at https://sv-comp.sosy-lab.org/2026/results/results-verified/C.unreach-call.Concurrency.table.html#/table?filter=id_any(value(exponential)).

initially: `x:=y:=z:=0`

Process p_0	Processes $p_1, ..., p_N$

`repeat N times:`
`  z := z+2*y`
`if z mod 2 = 0:`
`  x := 0`
`else:`
`  x := 1`
	`y := y+1`

	Processes $p_{N+1}, ..., p_{2N}$ `y := y*y`

finally: `assert(x*y=0)`

Fig. 1.2: Motivational example for possible exponential gain

interleavings. However, our algorithm notices that we do not track any information about `y`, so it will not consider the actions using `y` dependent. This way, our method explores a single interleaving and proves the safety of the program.

Summarizing our contributions: we introduce a general abstraction-aware partial order reduction approach which is independent of both the underlying POR algorithm and the abstract domain and which uses the information about the abstraction to boost partial order reduction. By proving necessary and sufficient conditions for the correctness of partial order reduction using the abstraction-aware dependency relation, we also prove that using the traditional dependency relation for partial order reduction in an abstract state space is correct as well (which is also a non-trivial statement as testified by Example 1.1). We have implemented and evaluated the proposed method in the abstraction-based verification tools Theta [42] and `CPAchecker` [11].

The organization of the paper is as follows. First, Section 2 introduces the theoretical background. We discuss the definitions and properties of dependency relations in Section 3. We present the general approach to combine POR and abstraction in an abstract state space exploration in Section 4. In Section 5, we demonstrate an actual partial order reduction algorithm implementing the general abstraction-aware extension to POR. Finally, we evaluate the approach in Section 6 using the algorithm introduced in Section 5 and another algorithm implementing the proposed idea whose detailed description and proof is omitted from this paper (since it would only complicate the presentation and would not carry additional insight regarding the proposed method).

2 Preliminaries

This section introduces the preliminary knowledge that is necessary to understand the methods described in the paper.

2.1 Representation

In this paper, we assume a computation model of concurrent programs, where processes (threads) communicate via shared variables. We assume a sequential consistency memory model. Though it would be easy to incorporate extra features into the model (such as heap memory, dynamic thread creation or termination, and synchronization primi-

tives), we strive to keep the presentation simple and skip these details. Our evaluation implementation naturally supports these features.

Concurrent programs are represented by multi-threaded control-flow automata (CFA) [41]:

Definition 2.1 A multi-threaded CFA is a tuple (V, P), where

- V is a set of (global) variables,
- P is a set of processes. A process is a tuple $p = (L, l_0, A, E)$, where:
  - L is a set of control locations with $l_0 \in L$ as the initial location,
  - A is a set of statements,
  - $E \subseteq L \times A \times L$ is a set of transitions. A transition is a directed edge with a source control location, a target control location, and a statement.

Each variable $v \in V$ has a domain D_v (the possible values for v), and possibly an initial value from its domain. A statement can be a deterministic assignment (`v := expr`), a non-deterministic assignment (`havoc v`) where the new value of v can be anything from its domain, or a guard condition (`[cond]`). For the verification of reachability properties, some CFA locations are marked as error locations: the program is safe if no error location can be reached by any of its processes.

We define transition systems (state spaces) as follows:

Definition 2.2 A transition system is a tuple (S, A, T, I), where S is a set of states, A is a set of actions, $T \subseteq S \times A \times S$ is a set of transitions, and $I \subseteq S$ is a non-empty set of initial states.

An action α is an *enabled* action from a state s if there is a transition $(s, \alpha, s') \in T$ for some $s' \in S$. We use the following notations:

- $s \xrightarrow{\alpha} s'$ denotes the transition (s, α, s'),
- $\alpha(s) = \{s' \in S : \quad \exists (s, \alpha, s') \in T\}$,
- $\mathit{enabled}(s)$ denotes the set of enabled actions from s,
- $\mathit{written}(\alpha)$ and $\mathit{read}(\alpha)$ are the sets of variables written/read by α, respectively,
- $\mathit{vars}(\alpha) = \mathit{write}(\alpha) \cup \mathit{read}(\alpha)$.

The state space of a program is a transition system where a state stores the CFA locations of all processes and the values of all variables. An action of a transition corresponds to a statement of a single process (processes step asynchronously). We use the Greek alphabet for actions and write p_α for the process of action α. A state is an error state if any of the processes is in an error location in the state. We denote the control location of process p in state s by $s(p)$, and the value of the variable v in state s by $s(v)$. For the states s_1, s_2, and the set of variables $U \subseteq V$, we write $s_1 =^U s_2$ to denote that $s_1(v) = s_2(v)$ for each $v \in U$. We define an expression function for a state s based on the values of variables in s: $\mathit{expr}(s) := \bigwedge_{v \in V}(v = s(v))$.

By $w = t_1 ... t_k$, we denote a transition sequence (or trace) and use the following for the concatenation of transition sequences or transitions: $w.v$. The notation $w \backslash t$ refers to the transition sequence obtained from w by removing the first occurrence of t from w. We often use actions in notations instead of transitions: an action α used in such a context means a transition with α as its action. We also refer to action sequences as traces. If we have a trace from a state that leads to an error state, we call this trace an error trace.

A transition system is action-deterministic if $|I| \leq 1$ and $|\alpha(s)| \leq 1$ for any state $s \in S$ and action $\alpha \in A$ [7]. The state space of a program is not action-deterministic due to non-deterministic assignments, and uninitialized variables. However, *unknown* is a possible value for variables when using abstraction (see details in Section 2.2): an uninitialized variable or a variable after a non-deterministic assignment gets the specific value *unknown*. This way, the state space becomes action-deterministic. In an action-deterministic transition system, we use $\alpha(s)$ for the single state s' with $(s, \alpha, s') \in T$. Partial order reduction algorithms are classically formulated for action-deterministic transition systems [6, 7, 22]. In some cases, instead of using the term action-deterministic, it is said that control non-determinism is allowed [3]. Some works investigate partial order reduction algorithms in non-deterministic state spaces, also considering types of abstraction introducing non-determinism [28]. In those settings, partial order reduction algorithms need to satisfy different properties. However, this research direction is out of this work's scope, so we assume that the abstract state space is action-deterministic.

A further assumption that we inherit from partial order reduction algorithms that we improve in this chapter is that the state space is finite. For data variables, this is not a real restriction in most cases, as the domain of program variables are typically bounded (e.g., 32-bit integers in most languages) or abstraction can take care of a finite domain (e.g., the size of the domains of each predicate in predicate abstraction is 2). The restriction concerns concurrent programs where new threads are created dynamically in an infinite loop. Naturally, in real scenarios, we do not need an unbounded number of threads (and in fact, the limitations of the execution environment also limits the possible number of running threads at the same time), so this is also only a theoretical limitation. Besides, having a finite state space is a common assumption to make in papers presenting partial order reduction algorithms [2, 24, 43].

2.2 Abstraction

An abstraction can be defined with an abstract domain, a precision, and a transfer function [10]. In this paper, we use a simplified definition of the abstract domain:[2]

Definition 2.3 An abstract domain is a tuple (S, expr), where S is a set of abstract states, and $\mathit{expr} : S \mapsto \mathit{FOL}$ is an expression function mapping an abstract state to a first-order logic formula describing the state.

We assume that CFA locations of all processes are explicitly tracked in all abstract domains: we refer to the location of process p in an abstract state s by $s(p)$. An abstract state s represents a concrete state c denoted by $c \models s$ if $c(p) = s(p)$ for each process p, and $\mathit{expr}(c) \implies \mathit{expr}(s)$. In the rest of the paper, we mainly use s for abstract states and c for concrete states. An abstract state is an error state if any process is in an error location. An abstract trace $w = \alpha_1 ... \alpha_k$ from the abstract state s_0 ($s_0 \xrightarrow{\alpha_1} ... \xrightarrow{\alpha_k} s_k$) is *feasible* if w is also a trace in the concrete state space ($c_0 \xrightarrow{\alpha_1} ... \xrightarrow{\alpha_k} c_k$) with $c_i \models s_i$; otherwise, w is spurious from s_0. If w is feasible, the sequence of concrete states $c_0, ..., c_k$ is a *concretization* of w from s_0. The abstract state space over-approximates the behavior of the concrete state space: if there is a

[2] Abstract states are usually defined as a semi-lattice with a partial order [10], but we do not need those details for this paper, so we simplify.

trace w from a concrete state c, then w is also a trace in the abstract state space from all abstract states s with $c \models s$ [10].

The precision describes which aspects of the model the abstraction tracks, defined differently for each abstract domain. The *variables of a precision* $vars(\Pi)$ are the variables that may appear in abstract state expression formulae. That is, the abstraction tracks no information about variables in $V \setminus vars(\Pi)$. The transfer function calculates the successor states of an abstract state with respect to a statement and a precision. We briefly explain two frequently used abstract domains: explicit-value abstraction [12] and predicate abstraction [23].

Explicit-value abstraction. In explicit-value abstraction, an abstract state is defined by the CFA locations of processes and an abstract variable assignment. The domain D_v of each variable $v \in V$ is extended with a top value $\top_v$. The top value represents an unknown value for the variable. The precision in explicit-value abstraction is the subset of variables $\Pi \subseteq V$ that are explicitly tracked in this abstraction; $vars(\Pi) = \Pi$. The values of other variables are unknown in all abstract states: $s(v) = \top_v$ for each $v \in V \setminus \Pi$. The expression function is $expr(s) := \bigwedge_{v \in V, s(v) \neq \top_v} (v = s(v))$. The result of the transfer function is based on the strongest post-operator under abstract variable assignment [12].

Predicate abstraction. In predicate abstraction, an abstract state is defined by the CFA locations of processes and a combination of FOL predicates. The precision is the set of FOL predicates (e.g., `x > 0`, `y = z`) that are tracked in this abstraction; $vars(\Pi)$ is the set of variables appearing in the tracked predicates. The expression function of an abstract state is the combination of FOL predicates that describes the state. The transfer function returns the strongest combination of predicates in the precision that is entailed by the source state and the operation.

Counterexample-Guided Abstraction Refinement (CEGAR) [17] is an abstraction-based model checking algorithm. CEGAR starts from a coarse abstraction and iteratively refines the abstraction until it can prove or disprove the analyzed property. The core of the algorithm is the *CEGAR-loop* which consists of the *abstractor* and the *refiner*. The abstractor builds the abstract state space (an *abstract reachability graph*, ARG [9]) over an abstract domain with a given precision. Since the abstract state space is an over-approximation of the original concrete state space, if no abstract error state is reachable, the concrete model is also safe. On the other hand, when an abstract error is reachable, the refiner checks whether it is a *feasible* or a *spurious* abstract counterexample. The counterexample is an alternating sequence of abstract states and actions from the initial abstract state to an abstract error state. The refiner checks whether this trace is feasible or not, that is, whether there is a concrete variable assignment for each state of the trace that does not contradict the abstract state expressions and the actions of the trace. If feasible, the model is unsafe. If spurious, the precision is refined. The abstract state space is built with this refined precision in the next iteration.

2.3 Partial Order Reduction

Partial Order Reduction (POR) is a well-known technique to avoid the exploration of redundant thread interleavings in the verification of a multi-threaded program [24]. Its key idea is to define an equivalence relation on traces and explore a single representative

(or as few as possible) of each equivalence class. Traces are defined to be equivalent if they can be obtained from each other by successively swapping adjacent *independent* actions. An equivalence class is called a *Mazurkiewicz trace* [33]. Intuitively, if adjacent independent actions are swapped, the outcome will remain the same: by exploring a single trace from each such equivalence class, we still cover all behaviors of the system. For the above interpretation of equivalence, we need a definition of independence.

Dependency plays a key role in partial order reduction. The general formulation of dependency is as follows [24]:

Definition 2.4 (Valid Dependency Relation) Let A be a set of actions. Let $D \subseteq A \times A$ be a binary, reflexive, and symmetric relation. D is a *valid dependency relation* if for all $\alpha, \beta \in A$, $(\alpha, \beta) \notin D$ (α and β are *independent*) implies that the following two conditions hold for all $s \in S$:

- if $\alpha \in enabled(s)$, then $\beta \in enabled(s)$ iff $\beta \in enabled(\alpha(s))$, *and*
- if $\alpha, \beta \in enabled(s)$, then $\beta(\alpha(s)) = \alpha(\beta(s))$.

α and β are *dependent* if they are not independent.

The first condition means that independent actions can neither disable nor enable each other. The second property states that independent actions commute. Sometimes, *dependency of transitions* is used in this paper: by the dependency of transitions, we mean dependency of their actions. Note that we will introduce further relations in this work that are not *valid* dependency relations; however, for easier understanding, we will refer to them as dependency relations – without the label *valid*.

Since the goal of POR is to avoid exploring multiple traces leading to the same state, the definition cannot be used directly for determining dependency (two actions should be explored in both orders to decide whether they commute). One of the main questions of this paper is how we can appropriately approximate the dependency relation in an abstract state space. Many algorithms have been introduced for partial order reduction in the last decades. Our abstraction-aware extension is orthogonal to the underlying POR algorithm. In this paper, we build the presentation on the concept of source sets which is the core concept behind optimal dynamic partial order reduction [1].

3 Dependence in Abstract State Spaces

This section studies commutativity, the core ingredient of partial order reduction in an abstract state space. As we demonstrated in Section 1, it is not trivial that the traditional concept of dependency relations in concrete state spaces transfers to abstract state spaces. In this section, we introduce different dependency relations and study their important properties that we can use in the rest of the paper to prove the correctness of the presented partial order reduction approach. Although most of the concepts presented in this section are not novel, we present the topic from a different angle compared to existing works [21, 38] that is better suited to our proposed approach.

3.1 Dependency relations

In our algorithms, different dependency relations are used for partial order reduction. It is important to note that these are not necessarily valid dependency relations in all transition systems. First, we define the syntactic dependency relation.

3.1.1 Syntactic Dependency Relation

A syntactic dependency relation denoted by D_S is the classically used syntactic over-approximation of dependency [24]. That is, two actions $(\alpha, \beta) \in D_S$ (α and β are dependent) iff:

- α and β are actions of the same process, *or*
- $vars(\alpha) \cap vars(\beta) \neq \emptyset$, and at least one variable in $vars(\alpha) \cap vars(\beta)$ is written by α or β.

The syntactic dependency relation is a valid dependency relation in the concrete state space [24]. However, in general, it is not a valid dependency relation in the abstract state space; see the motivating example Example 1.1.

3.1.2 Abstraction-Aware Dependency Relation

We introduce a new dependency relation for abstract state spaces (similar conditions are used in [21]). An abstraction-aware dependency relation is also a syntactic approximation. However, it is defined in an abstraction with respect to the precision of the abstraction.

Definition 3.1 Let us have an abstract state space built with precision Π, and let D_Π be a binary, reflexive, and symmetric relation. Two actions $(\alpha, \beta) \in D_\Pi$ (α and β are dependent with respect to precision Π) iff:

- α and β are actions of the same process, *or*
- $vars(\alpha) \cap vars(\beta) \cap vars(\Pi) \neq \emptyset$, and at least one variable in $vars(\alpha) \cap vars(\beta) \cap vars(\Pi)$ is written by α or β.

In other words, the second condition means that α and β may still be independent if they use common variables. They are only dependent when the abstraction tracks any information about any variable that they both access. Note that D_Π is a subset of the syntactic dependency relation D_S for any precision Π: the first condition is the same for both relations, and the second condition of D_Π implies the second condition of D_S. As a consequence, D_Π is not a valid dependency relation in the abstract state space either (same counterexample from Example 1.1). Furthermore, D_Π is not necessarily a valid dependency relation in the concrete state space.

Example 3.1 To illustrate the difference between the syntactic and the abstraction-aware dependency relation, consider the actions `x:=1` and `[x>0]`. They are dependent based on the syntactic dependency relation since they both use the variable x, and the first action writes it. For the abstraction-aware dependency relation, we need an abstraction with a precision. First, let the precision be $\Pi = \{x < y,\ z = 1\}$: then the two actions are dependent in D_Π since we have information about x in this abstraction. However, when the precision is $\Pi = \{0 < y,\ z = 1\}$, then our actions are independent in D_Π as the value of x is completely ignored in this abstraction.

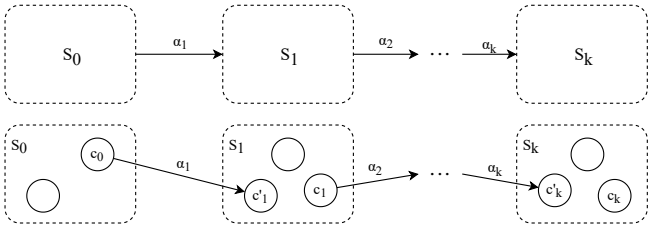


Fig. 3.1: Abstract trace with a partial concretization

As we have seen, the introduced relations are not valid dependency relations. However, we show in the next sections that using these relations (D_{Π} in particular) for partial order reduction in an abstract state space with precision Π, feasible errors are still found.

3.2 Partial Feasibility

First, we generalize the concept of abstract trace feasibility by introducing partial feasibility. Intuitively, a partial concretization of an abstract trace is a partial variable assignment for each abstract state of the trace with only a subset of variables assigned so that the partial variable assignment does not contradict the abstract state expressions. The motivation for introducing partial feasibility is that variables ignored in the abstraction may spoil feasibility.

Example 3.2 Let us have a program with the following three independent actions (with the variables initially being zero):

α: `x:=1` | β: `[y=0] reach error` | γ: `[z=1] z:=0`

Clearly, γ is not enabled in the initial concrete state due to z being zero. Let us have the abstraction where only the variables x and y are tracked explicitly. Now, γ is enabled in the abstract initial state as we have no information about z. A partial order reduction algorithm using the abstraction-aware dependency relation may explore the trace $w = \gamma.\alpha.\beta$ and no other traces as all actions are independent. Even though w is not feasible, it will be enough for us in a sense that is formalized by partial feasibility.

Definition 3.2 An abstract trace $w = \alpha_1 ... \alpha_k$ from an abstract state s_0 ($s_0 \xrightarrow{\alpha_1} ... \xrightarrow{\alpha_k} s_k$) is *partially feasible* for the set of variables $P \subseteq V$ if there are concrete states $c_0, ..., c_k$ such that:

- $c_i \models s_i$ for each $0 \leq i \leq k$, and
- for each $0 \leq i < k$, $\exists c'_{i+1}$ such that $c_i \xrightarrow{\alpha_{i+1}} c'_{i+1}$, $c'_{i+1}(p) = c_{i+1}(p)$ for each process p, and $c'_{i+1} =^P c_{i+1}$.

We refer to $c_0, ..., c_k$ as a *partial concretization* of w for P.

If w consists of a single action α, we say that α is a partially feasible action from the abstract state. Note that w being partially feasible for V (the set of all variables) means that w is feasible. The connection between feasibility and partial feasibility is straightforward: if w is not partially feasible for a $P \subseteq V$, then w is not feasible;

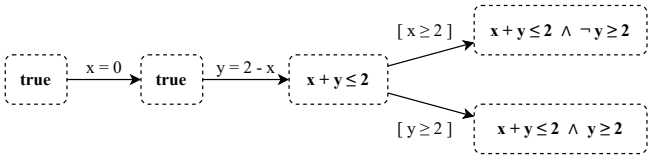


Fig. 3.2: Example of a not partially feasible abstract trace

and if w is feasible, then w is partially feasible for any $P \subseteq V$. In practice, partial feasibility for $P = vars(\Pi)$ will be relevant for us where Π is a precision. Thus, partial feasibility will only depend on variables that we have information about in the current abstraction. An alternative to introducing partial feasibility would be to use statement abstraction [21]: instead of relaxing the concept of feasibility to accommodate the case of untracked variables, we could abstract the statements themselves to erase the parts affecting untracked variables. However, we find it more intuitive to keep the abstract state space as it is, and extend the notion of feasibility.

Example 3.3 Figure 3.1 illustrates partial feasibility: we have the abstract trace $\alpha_1 ... \alpha_k$ from the abstract state s_0 and it has a partial concretization $c_0, ..., c_k$ (for the sake of simplicity, we omitted the values of variables from the figure).

We can see an example of an abstract trace that is not partially feasible for $\{x, y\}$ in Figure 3.2. The figure shows an abstract state space built with a precision consisting of the predicates $x + y \leq 2$ and $y \geq 2$. The state expression of the initial state is *true*, and it remains *true* even after executing $x = 0$ since none of the predicates or their negated form is a consequence of this action. However, $y = 2 - x$ implies $x + y \leq 2$. It still allows both $[x \geq 2]$ and $[y \geq 2]$ to be enabled. Even though $x = 0$, $y = 2 - x$, $[x \geq 2]$ is an abstract trace from the initial state, it is trivially not partially feasible since there is no possible partial variable assignment for $vars(\Pi) = \{x, y\}$ meeting the requirements of Definition 3.2 due to x. Also note that the actions $[x \geq 2]$ and $[y \geq 2]$ (belonging to different processes) disable each other even though they are independent based on the syntactic dependency relation.

The following lemma states that if we have a sequence of concrete states $c_0, ..., c_k$ that satisfy the conditions in Definition 3.2, then there is indeed an abstract trace in the abstract state space whose partial concretization is the sequence $c_0, ..., c_k$.

Lemma 3.1 *Let Π be the precision of the abstraction and $P = vars(\Pi)$; and $c_0, ..., c_k$ be concrete states such that for each $0 \leq i < k$, $\exists c'_{i+1}$ with $c_i \xrightarrow{\alpha_{i+1}} c'_{i+1}$, $c'_{i+1}(p) = c_{i+1}(p)$ for each process p, and $c'_{i+1} =^P c_{i+1}$. Let s_0 be any abstract state with $c_0 \models s_0$. Then: $\alpha_1 ... \alpha_k$ is an abstract trace that exists in the abstract state space from s_0.*

Proof. The abstract state space being an over-approximation of the concrete state space means that if there is a transition $c_i \xrightarrow{\alpha_{i+1}} c'_{i+1}$ in the concrete state space, then there is a transition $s_i \xrightarrow{\alpha_{i+1}} s_{i+1}$ in the abstract state space where $c_i \models s_i$ and $c'_{i+1} \models s_{i+1}$. The expression function of abstract states only uses the variables in the precision. Thus, from $c'_{i+1}(p) = c_{i+1}(p)$ for each process p, and $c'_{i+1} =^P c_{i+1}$, it follows that $c'_{i+1} \models s_{i+1}$ implies that $c_{i+1} \models s_{i+1}$. Now, with induction for i from 0 to $k - 1$, we get that $\alpha_1 ... \alpha_k$ is an abstract trace in the abstract state space from s_0. □

3.3 Relaxed Partial Order Representation

We can connect the partial order representation induced by a dependency relation with partial feasibility. Let $w_1 \approx^{\Pi} w_2$ denote that abstract traces w_1 and w_2 can be transformed into each other by successively swapping adjacent actions that are independent in D_{Π}. Thus, the relation $\approx^{\Pi}$ defines equivalence classes (Mazurkiewicz traces [33]) on abstract traces. The following theorem states that either all abstract traces are partially feasible in such an equivalence class or none.

Theorem 3.1 *Let us have an abstract state space S built with precision Π, let $s \in S$ be an abstract state, w_1 and w_2 be transition sequences with $w_1 \approx^{\Pi} w_2$, and $P = vars(\Pi)$.*

Then, w_1 is a partially feasible abstract trace from s for P iff w_2 is a partially feasible abstract trace from s for P.

Proof. We show that swapping two adjacent actions independent based on D_{Π} in a partially feasible abstract trace w will result in an abstract trace w' that is also partially feasible. The case is symmetric for w_1 and w_2: we can assume that w_1 is partially feasible. Since $w_1 \approx^{\Pi} w_2$ means that the partially feasible abstract trace w_1 can be transformed into w_2 by successively swapping adjacent independent actions, and partial feasibility is preserved in each step, it follows that w_2 is partially feasible.

Let $w = q.\alpha.\beta.r$ for some traces q and r ($s \xrightarrow{q} s_q \xrightarrow{\alpha} s_\alpha \xrightarrow{\beta} s_{\alpha\beta} \xrightarrow{r} s_w$) and $w' = q.\beta.\alpha.r$ from state s with w being partially feasible from s for P, and $(\alpha, \beta) \notin D_{\Pi}$. We check that w' is also partially feasible from s for P. Let $A = vars(\alpha)$, and $B = vars(\beta)$.

Since w is partially feasible, we have a partial concretization $c, ..., c_q, c_\alpha, c_{\alpha\beta}, ..., c_w$ of w with $c_q \models s_q$, $c_\alpha \models s_\alpha$, $c_{\alpha\beta} \models s_{\alpha\beta}$. Ignoring the end of w, we get that $c, ..., c_q, c_\alpha, c_{\alpha\beta}$ is a partial concretization of $q.\alpha.\beta$ for P. Our goal is to show that there is a partial concretization $c, ..., c'_q, c_\beta, c_{\beta\alpha}$ of $q.\beta.\alpha$ such that $c_{\alpha\beta} = c_{\beta\alpha}$.

Throughout the proof, we compare the values of variables in different concrete states. For this, as a reference, let us make the following observations:

Observation 1. *Since $(\alpha, \beta) \notin D_{\Pi}$, $A \cap B \cap P = \emptyset$ or neither α nor β modifies any variable in $A \cap B \cap P$. In both cases, $c_1(v) = c_2(v)$ for each variable $v \in A \cap B \cap P$ for any concrete states c_1, c_2 with $c_1 \xrightarrow{\alpha} c_2$ or $c_1 \xrightarrow{\beta} c_2$.*

Observation 2. *The action α can only change the values of variables in A, so if we have concrete states c_1 and c_2 with $c_1 \xrightarrow{\alpha} c_2$, then $c_1 =^{\overline{A}} c_2$. Similarly for β.*

In Figure 3.3, we visualize the concrete and abstract states appearing in the proof. The variable values are represented by Venn diagrams in each concrete state. Sets with the same colors (and numbers) indicate that variables belonging to them have the same values. The colored numbers in the text should be interpreted such that the set of variables mentioned just before the colored numbers is the union of sets represented by the numbers.

Using Definition 3.2 of partial feasibility for the partial concretization $c, ..., c_q, c_\alpha, c_{\alpha\beta}$: there is a concrete state c'_α such that $c_q \xrightarrow{\alpha} c'_\alpha$ and $c'_\alpha =^{P} c_\alpha$ 1 2 4 10 ; similarly, there is a concrete state $c'_{\alpha\beta}$ such that $c_\alpha \xrightarrow{\beta} c'_{\alpha\beta}$ and $c'_{\alpha\beta} =^{P} c_{\alpha\beta}$ 1 2 10 13 .

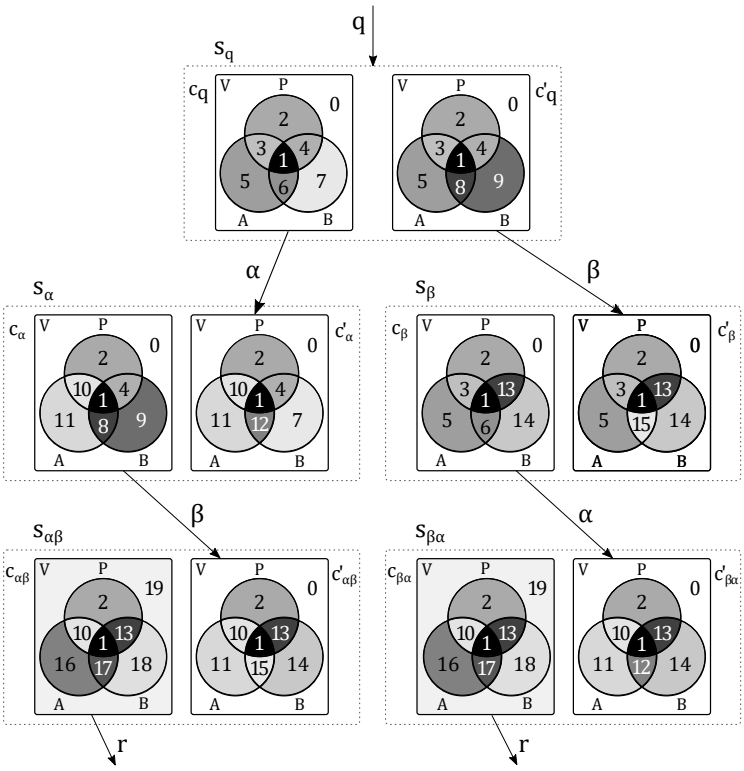


Fig. 3.3: Variables of states in the proof of Theorem 3.1.

Based on Observation 2 for α, $c_q = c'_\alpha$ on $\overline{A}$ $\boxed{0}$ 2 4 7. Putting it together, $c_q =^{P\setminus A} c_\alpha$ 2 4. Let c'_q such that $c'_q(p) = c_q(p)$ for each process p[3], and $c'_q =^{\overline{B}\cup P} c_q$ $\boxed{0}$ 1 2 3 4 5. Also, let $c'_q =^{B\setminus P} c_\alpha$ 8 9. Defining c'_q this way, with Observation 1 we can conclude that $c'_q =^{B} c_\alpha$ 1 4 8 9.

From partial feasibility, we know that β is enabled in c_α. The guard condition of β only depends on the values of variables in B. Therefore, by the fact that $c'_q =^{B} c_\alpha$, β is enabled in c'_q: there is a concrete state c'_β with $c'_q \xrightarrow{\beta} c'_\beta$.

Let c_β be a concrete state with $c_\beta =^{\overline{A}\cup P} c'_\beta$ $\boxed{0}$ 1 2 3 13 14. Also, let $c_\beta =^{A\setminus P} c_q$ 5 6. Using the definitions of c_β and c'_q, and Observation 2 for β: $c_\beta =^{(A\cap P)\setminus B} c'_\beta =^{(A\cap P)\setminus B} c'_q =^{(A\cap P)\setminus B} c_q$ 3. By the definition of c_β and Observation 1, $c_\beta =^{A} c_q$ 1 3 5 6. This way, the fact that $\alpha \in \mathit{enabled}(c_q)$ implies that α is enabled in c_β.

Now, let $c'_{\beta\alpha}$ be such that $c_\beta \xrightarrow{\alpha} c'_{\beta\alpha}$. Let $c_{\beta\alpha}$ be such that $c_{\beta\alpha} =^{P} c'_{\beta\alpha}$ 1 2 10 13. Also, let $c_{\beta\alpha} =^{\overline{P}} c_{\alpha\beta}$ 16 17 18 $\boxed{19}$. We show that $c_{\beta\alpha} =^{P} c_{\alpha\beta}$ 1 2 10 13.

[3] For better readability, we omit CFA locations from now on. In our notation, states with the same name differing only in an apostrophe have the same CFA locations for each process.

Again, consider that $c_q =^A c_\beta$ 1 3 5 6, thus α from c_q and c_β will yield the same new values for each $v \in A$: $c'_\alpha =^A c'_{\beta\alpha}$ 1 10 11 12. From Observation 2 for β, and the definitions of the following states, $c_{\alpha\beta} =^{A\cap P} c'_{\alpha\beta} =^{A\cap P} c_\alpha =^{A\cap P} c'_\alpha =^{A\cap P} c'_{\beta\alpha} =^{A\cap P} c_{\beta\alpha}$ 1 10. Symmetrically, $c_{\alpha\beta} =^{B\cap P} c'_{\alpha\beta} =^{B\cap P} c'_\beta =^{B\cap P} c_\beta =^{B\cap P} c'_{\beta\alpha} =^{B\cap P} c_{\beta\alpha}$ 1 13. Finally, $c_{\beta\alpha} =^{P\setminus(A\cup B)} c_q =^{P\setminus(A\cup B)} c_{\alpha\beta}$ on, since neither α, β, nor the definition of any concrete state changed the value of such a variable v 2.

Thus, $c_{\beta\alpha} =^V c_{\alpha\beta}$ (for all variables). It can be easily seen that process locations are the same in $c_{\beta\alpha}$ and $c_{\alpha\beta}$, so $c_{\beta\alpha} = c_{\alpha\beta}$. Based on the above definitions of c_β and $c_{\beta\alpha}$, the property proven above that $c_{\beta\alpha} = c_{\alpha\beta}$, and using Lemma 3.1 for the sequence of concrete states $c, ..., c_q, c_\beta, c_{\beta\alpha}, ..., c_w$, we obtain that $w' = q.\beta.\alpha.r$ is a partially feasible abstract trace for P from the abstract state s which proves the theorem. □

Note: relating the claim of Theorem 3.1 to the work of *Farzan et al.* [21], it corresponds to their statement that a commutativity relation induced by a projection to the proof (similar to precision in our notion) is safe with respect to the proof[4].

4 Abstraction-Aware Partial Order Reduction

This section presents how we can use the abstract dependency relation for partial order reduction in an abstract state space exploration algorithm. Since the core concept of the approach is to consider additional information about the abstraction used when applying partial order reduction, we call the algorithm *abstraction-aware partial order reduction*. Our approach is independent of the underlying POR algorithm as well as the applied abstract domain. The only requirements for the abstract domain are that CFA locations of all processes must be explicitly tracked, and the abstract state space must be action-deterministic (that is, the transfer function must return a single successor for each state and operation) to conform with existing partial order reduction algorithms.

Having partial feasibility in our toolbox, we can establish the necessary condition that any sound partial order reduction algorithm applied in an abstract state space must satisfy. First, we identify in Section 4.1 on a high-level what is expected from any partial order reduction in an abstract state space to be considered correct. Then, we formalize this condition in Section 4.2. Finally, we also prove a sufficient condition for the correctness of POR in an abstract state space in Section 4.3.

[4] Although these two claims are equivalent in the sense that they could be easily proved based on each other, the two proofs can show different insights: ours is directly focused on the viewpoint of exploring the abstract state space referring only to the original statements, while theirs is focused on how a proof of infeasibility generalizes between traces when using abstracted statements. While this result is sufficient for soundness in the context of their proof-checking algorithm [20], proving the soundness of our abstract state space exploration algorithm also involves proving that we can lose representatives only for infeasible traces, as shown in the next section. Their algorithm generates a representative for every Mazurkiewicz-trace ending in an accepting state, even infeasible ones, whereas we may prune infeasible traces earlier and thus allow losing representatives of some Mazurkiewicz-traces.

Contains feasible trace	✓	✓	✗	✗
Contains infeasible trace	✗	✓	✓	✓
Partially feasible for $vars(\Pi)$	✓	✓	✓	✗
Example	x:=1 y:=1 y:=1 x:=1	y:=1 [y=1] [y=1] y:=1	[y=1] [z=1] [z=1] [y=1]	[x=1] [y=1] [y=1] [x=1]

Fig. 4.1: Classification of abstract Mazurkiewicz-traces assuming $vars(\Pi) = \{x\}$ in the abstraction and all variables are zero in the initial states. Feasible traces in green, infeasible ones in red.

4.1 Correctness Criterion of Abstraction-Aware Partial Order Reduction

So far, we have defined equivalence classes (abstract Mazurkiewicz-traces) on abstract traces, with the abstract dependency relation D_Π. Evidently, if a partially feasible abstract trace reaches an error state, then all other traces in its equivalence class reach an error state. This comes from the fact that error states are defined as states in which any process is in an error location. Since all traces of an equivalence class contain the same actions (cf. they can be obtained from each other by successively swapping certain actions), if an action leads to an error location, it will reach the error in all traces of the equivalence class regardless of the order of actions. Therefore, partial order reduction algorithms traditionally require exploring (at least) one representative of each equivalence class to ensure correctness [2, 24]. In the concrete case, traces belonging to the same Mazurkiewicz-trace are either all feasible or all infeasible by the definition of concrete commutativity. Therefore, partial order reduction algorithms without abstraction implicitly assume to only speak about Mazurkiewicz-traces with feasible traces. The case is more complicated with abstraction: there can be abstract Mazurkiewicz-traces that contain both feasible and infeasible traces.

Take the classification of equivalence classes provided in Figure 4.1 based on the feasibility and partial feasibility (for the variables of the precision) of traces in an equivalence class. Based on Theorem 3.1, all traces belonging to the same equivalence class are either partially feasible or partially infeasible. However, both feasible and infeasible traces can belong to the same abstract Mazurkiewicz-trace. This gives us three classifying factors for the equivalence classes: whether they contain feasible and infeasible trace, and the partial feasibility of all traces in the class. Since a feasible trace is also partially feasible for any subset of variables, there is no such class that contains feasible traces, but the traces in the class are not partially feasible. Furthermore, empty equivalence classes are not interesting (i.e., ones neither containing feasible nor infeasible traces, be they partially feasible or not). Thus, we end up with the four classes of abstract Mazurkiewicz-traces summarized in Figure 4.1.

On the highest level, it is required from an abstract state space reduction that if an error state can be reached in the original concrete state space, then an error state is also reachable in the reduced abstract state space; this is to ensure that reachability analysis performed in the reduced abstract state space does not miss any feasible errors. Therefore, it is reasonable to relax the correctness criterion of a partial order reduction algorithm to only require the exploration of (at least) one trace from equivalence classes

that contain feasible traces. Naturally, there is no need to discover infeasible error traces even if they are present in the abstract state space since the final verification verdict is based on the existence of a feasible error trace. In other words, if abstract error states can only be reached with spurious traces (meaning that there is no error state in the concrete state space), there is no need to include an error state in the reduced state space.

That is, the correctness criterion of a partial order reduction algorithm using the abstraction-aware dependency relation is to explore at least one representative from each abstract Mazurkiewicz-trace belonging to the first two classes in Figure 4.1 where the equivalence class contains feasible traces. Unfortunately, practically, we cannot distinguish between feasible and infeasible traces in the abstract state space. We can only decide partial feasibility with respect to the variables of the precision. So, practically, the correctness criterion for POR becomes the following: at least one representative from each abstract Mazurkiewicz-trace containing partially feasible traces for $vars(\Pi)$ must be explored. We formalize this intuitive requirement in the next section.

Note that exploring a single representative from each abstract Mazurkiewicz-trace containing partially feasible traces for the precision variables only guarantees that we find an abstract error if there is an error in the concrete state space. It is not guaranteed that the single explored representative of a class is actually feasible. Naturally, it is an inherent property of abstraction-based model checking that the discovered abstract error traces can be spurious. A feasibility check and refinement of the abstraction may be necessary to confirm and obtain a feasible abstract error trace.

4.2 Source Sets for Abstraction-Aware Partial Order Reduction

The necessary condition for the correctness of any partial order reduction in a concrete state space has been formulated in the literature using the concept of *weak initials* and *source sets* [3]. We generalize this concept to the abstract case and reformulate the necessary condition for partial order reduction in the abstract state space using the abstraction-aware dependency relation. Using the relation $\approx^{\Pi}$ on abstract traces, we relax the definition of weak initials and source sets:

Definition 4.1 (Weak Initials) Let s be an abstract state, Π be the precision of the abstraction, and w be a transition sequence from s such that w is partially feasible from s for $vars(\Pi)$. For w, the set $WI_s^{\Pi}(w)$ of weak initials in s is a set of actions: $\alpha \in WI_s^{\Pi}(w)$ iff there are transition sequences v_1 and v_2 such that $\alpha.v_1 \approx^{\Pi} w.v_2$, and $w.v_2$ is partially feasible from s for $vars(\Pi)$.

Definition 4.2 (Source Sets) Let s be a state and W be a set of transition sequences such that w is an abstract trace from s for each $w \in W$. A set $S \subseteq enabled(s)$ is a *source set for W in s with respect to Π* if for each trace $w \in W$, $WI_s^{\Pi}(w) \cap S \neq \emptyset$.

For abstract traces w and w' from the abstract state s, we use $w \sqsubseteq_s w'$ to denote that $w.v \approx^{\Pi} w'$ for some transition sequence v. In such cases, we say that w' is an equivalent extension of w.

Example 4.1 To demonstrate weak initials and source sets, take the program of Figure 1.2, and let s_0 be the abstract initial state. Consider two precisions Π_1 and Π_2. In the first case, let us explicitly track all variables of the program, so $vars(\Pi_1) =$

$\{x, y, z\}$. In the other case, let us only track the predicates `z` mod `2 = 0` and `x = 0`, so $vars(\Pi_2) = \{x, z\}$.

Let w_1 be the following trace of the program: first, the actions of processes p_1, ..., p_{2N} in order (actions writing y), then the actions of p_0 (with the first branch of the *if*). Let w_2 be another trace, where the actions of p_0 come first, and then the actions of processes p_1, ..., p_{2N}. Evidently, both w_1 and w_2 are feasible traces from the initial state.

In the first case, the weak initials $WI_{s_0}^{\Pi_1}(w_1)$ only contains `y := y+1` (of p_1). To see this, note that no action α of the program is independent based on D_{Π_1} with all preceding actions of α in w_1 due to y being used by the first actions of all threads: it is impossible to consecutively swap adjacent independent elements to obtain another trace w_1' starting with α ($\nexists w_1'$ such that w_1' starts with α and $w_1' \approx^{\Pi_1} w_1$). Similarly, $WI_{s_0}^{\Pi_1}(w_2) = \{$`z := z+2*y`$\}$. Since w_1 and w_2 are both transition sequences from s_0, neither $P_0 = \{$`z := z+2*y`$\}$ nor $P_i = \{$`y := y+1`$\}$ nor $P_j = \{$`y := y*y`$\}$ (where P_is and P_js are the singleton sets containing the first actions of processes $p_1, ..., p_N$ and $p_{N+1}, ..., p_{2N}$, respectively) are source sets for all transition sequences in s_0, since $P_0 \cap WI_{s_0}^{\Pi_1}(w_1) = P_i \cap WI_{s_0}^{\Pi_1}(w_2) = P_j \cap WI_{s_0}^{\Pi_1}(w_2) = \emptyset$. In fact, the only source set in s_0 is $enabled(s_0)$ itself.

In the second case, actions using y are not dependent in D_{Π_2}. So $WI_{s_0}^{\Pi_2}(w_1)$ and $WI_{s_0}^{\Pi_2}(w_2)$ both contain all enabled actions in s_0 (the first actions of all threads can be swapped with independent swaps). In fact, $WI_{s_0}^{\Pi_2}(w) = enabled(s_0)$ for any partially feasible (for $vars(\Pi_2)$) abstract trace w starting from s_0. Therefore, any action $\alpha \in enabled(s_0)$ forms a source set $\{\alpha\}$ since α is part of the weak initials of all partially feasible traces starting from s_0: e.g., $\{$`z := z+2*y`$\}$ is a source set in s_0 for the set of partially feasible trace starting from s_0 with respect to the precision Π_2.

Using source sets, we can formulate the necessary condition for any correct partial order reduction algorithm in the abstract state space.

Property 4.1 Let Π be the precision of the abstraction, s be an abstract state, and W be the set of abstract traces starting from s that are partially feasible for $vars(\Pi)$. Let W' be a set of abstract traces from s such that for each $w \in W$ there is a $w' \in W'$ with $w \sqsubseteq_s w'$. Then, the set of first actions of traces in W' is a source set for W in s with respect to Π.

Here, W' represents the set of traces explored by a POR algorithm. Intuitively, we want W' to include an equivalent extension of all partially feasible traces which is captured by $w \sqsubseteq_s w'$. Interpreting Definition 4.1 of weak initials and the symbol $\sqsubseteq_s$ by using $w' = \alpha.v_1$ and $w.v_2 \approx^{\Pi} w'$ for some transition sequence v_2, we indeed obtain that α (the first action of w') is in $WI_s^{\Pi}(w)$. Thus, applying this to all partially feasible traces w from s, we meet the conditions in Definition 4.2 of source sets and obtain that the first actions of traces indeed form a source set for all partially feasible traces starting from s.

4.3 Sufficient Condition for the Correctness of Abstraction-Aware POR

We can also use this relaxed definition of source sets to formulate a sufficient condition for the correctness of the proposed abstraction-aware partial order reduction similarly to the sufficient condition in the traditional (non-abstract) case [3]. From a practical

viewpoint, the goal is to reduce the size of the abstract state space by exploring only a subset of enabled actions from each state. Reverting Property 4.1, we show that if the set of actions explored from each state form a source set with an additional condition for cycles in the abstract state space, it is guaranteed that all possibly feasible error traces have an equivalent extension among the explored traces.

Let $W_\Pi(s)$ denote the set of partially feasible traces from an abstract state s for $vars(\Pi)$ in an abstract state space with precision Π. The following theorem (a modified version of the corresponding one in [2]) states that a reduction based on source sets is also sufficient for the correctness of a partial order reduction algorithm.

Theorem 4.1 *Let A_O be the original abstract state space built with precision Π, and A_R be the reduced abstract state space obtained from A_O by restricting the set of actions that are explored from each state. If the following two conditions are satisfied:*

1. *for each state s in A_R, the set of explored actions is a source set for $W_\Pi(s)$ in s,*
2. *for each cycle in A_R, if an action α is enabled in all states of the cycle, then α is explored from some state of the cycle,*

then for each state $s \in A_R$ and abstract trace w from s in A_O such that w is partially feasible for $vars(\Pi)$, there is a transition sequence w' in A_R such that $w \sqsubseteq_s w'$.

The proof of the theorem uses the following lemma, so we include it before proving the theorem.

Lemma 4.1 *Let s be an abstract state, Π be the precision of the abstraction, and w be a trace from s such that w is partially feasible from s for $vars(\Pi)$. If $\alpha \in WI_s^\Pi(w)$ and $\alpha \notin w$, then*

1. *$w.\alpha$ is also partially feasible from s for $vars(\Pi)$, and*
2. *$\alpha.w \approx^\Pi w.\alpha$.*

Proof. From Definition 4.1 of weak initials, there are transition sequences v_1 and v_2 such that $\alpha.v_1 \approx^\Pi w.v_2$. This means that $w.v_2$ can be obtained from $\alpha.v_1$ by successively swapping adjacent independent actions. Since $\alpha \notin w$, $\alpha \in v_2$ necessarily. Also, for any action $\beta \in v_1$ but $\beta \notin w$, $\beta \in v_2$. Thus, any $\beta \notin w$ (including α) preceding any action $\gamma \in w$ in $\alpha.v_1$ must be independent with such γ actions: $(\beta, \gamma) \notin D_\Pi$.

So we can first move all such β after w by successive independent swapping steps without swapping α with any other such β. Thus, we get the transition sequence $w.\alpha.v_2'$ such that $\alpha.v_1 \approx^\Pi w.\alpha.v_2' \approx^\Pi w.v_2$. Since $w.v_2$ is partially feasible from s based on the definition of weak initials, $w.\alpha.v_2'$ is partially feasible as well by Theorem 3.1. Thus, its prefix $w.\alpha$ is partially feasible from s which proves the first statement.

For the second statement, take again that $(\alpha, \gamma) \notin D_\Pi$ for each $\gamma \in w$ since α precedes every other action in $\alpha.v_1$. This, by definition of the relation $\approx^\Pi$ means that $\alpha.w \approx^\Pi w.\alpha$. □

The proof of Theorem 4.1 proceeds similarly to the proof of the original theorem in [2], although some statements need more thorough justification.

Proof of Theorem 4.1. We prove the theorem by induction on the length of w. The base case with $|w| = 0$ is trivial. For the inductive step, let us have the trace $w \in W_\Pi(s)$: by definition of $W_\Pi(s)$, w is partially feasible from s. By condition (1), some action $\alpha \in WI_s^\Pi(w)$ is explored from s in A_R. We have two cases:

1. $\alpha \in w$
   By definition, $\alpha \in WI_s^{\Pi}(w)$ means that $\alpha.(w \backslash \alpha) \approx^{\Pi} w$ (otherwise, $\alpha.v_1$ could not be transformed into $w.v_2$ by successive independent swapping steps when applying Definition 4.1 for α and w). This implies together with the assumption that w is partially feasible from s that $\alpha.(w \backslash \alpha)$ is also a partially feasible abstract trace from s based on Theorem 3.1. As a consequence, $(w \backslash \alpha)$ is a partially feasible abstract trace for $vars(\Pi)$ from $\alpha(s)$.
   From the induction hypothesis for state $\alpha(s)$ and the partially feasible trace $(w \backslash \alpha)$ from $\alpha(s)$, we know that the reduced state space A_R contains a trace w'' with $(w \backslash \alpha) \sqsubseteq_{\alpha(s)} w''$. This way, A_R also contains the trace $\alpha.w''$ from s. We now have $\alpha.(w \backslash \alpha) \sqsubseteq_s \alpha.w''$. From this, along with $\alpha.(w \backslash \alpha) \approx^{\Pi} w$, we can easily infer from the definitions of $\approx^{\Pi}$ and $\sqsubseteq_s$ that $w \sqsubseteq_s \alpha.w''$, so we can take $\alpha.w''$ as w' in the theorem.
2. $\alpha \notin w$
   Let $\alpha_1 = \alpha$. Then, using Lemma 4.1, $\alpha_1 \in WI_s^{\Pi}(w)$, $\alpha_1 \notin w$, and the assumption about the partial feasibility of w imply that $w.\alpha_1$ is a partially feasible trace from s, and $\alpha_1.w \approx^{\Pi} w.\alpha_1$. This, together with Theorem 3.1 implies that $\alpha_1.w$ is also a partially feasible trace from s. Thus, w is a partially feasible trace from $\alpha_1(s)$.
   Again, by condition (1), some action $\alpha_2 \in WI_{\alpha_1(s)}^{\Pi}(w)$ is explored from $\alpha_1(s)$ in A_R. Continuing in this way, we have two cases:
   - There is a sequence of actions $\alpha_1, ..., \alpha_k$ such that for each $1 \leq i \leq k-1$, w is a partially feasible trace from $\alpha_{i-1}(...(\alpha_1(s)))$ with $\alpha_i \notin w$, and $\alpha_k \in w$, and for each $1 \leq i \leq k$, $\alpha_i \in WI_{\alpha_{i-1}(...(\alpha_1(s)))}^{\Pi}(w)$. Now, we can extend the reasoning in case 1 of the proof (with α_k being in a similar position as α in case 1): we have a trace w'' such that A_R contains the trace $\alpha_1...\alpha_k.w''$ from s such that $\alpha_1...\alpha_k.(w \backslash \alpha_k) \sqsubseteq_s \alpha_1...\alpha_k.w''$. Knowing that $\alpha_1...\alpha_k.(w \backslash \alpha_k) \approx^{\Pi} \alpha_k.(w \backslash \alpha_k).\alpha_1...\alpha_{k-1}$ implies $\alpha_k.(w \backslash \alpha_k) \sqsubseteq_s \alpha_1...\alpha_k.w''$. Since $\alpha_k.(w \backslash \alpha_k) \approx^{\Pi} w$, we have that $w \sqsubseteq_s \alpha_1...\alpha_k.w''$, so we can take $\alpha_1...\alpha_k.w''$ as w' in the theorem.
   - There is an unbounded sequence of actions $\alpha_1, \alpha_2, ...$ such that for each $i = 1, 2, ...$, w is a partially feasible trace from $\alpha_{i-1}(...(\alpha_1(s)))$, $\alpha_i \notin w$ but $\alpha_i \in WI_{\alpha_{i-1}(...(\alpha_1(s)))}^{\Pi}(w)$. Consequently, there must be a loop in this unbounded sequence of actions (since the state space if finite) with the first action w_1 of w enabled in all states of the loop. By condition (2), w_1 must be explored from at least one state of the loop, and we are back in case 1 of the proof.

□

If an error state is reachable in the concrete state space from the initial state with trace w, w is also an abstract trace from the abstract initial state s_0 leading to an abstract error state, since the abstract state space is an over-approximation of the concrete state space. As w is a feasible trace, it is also partially feasible for any subset of variables. As a consequence, Theorem 4.1 can be used for w: there is an abstract trace w' in the reduced abstract state space with $w \sqsubseteq_{s_0} w'$. Since error locations are sinks in the CFA, any state reachable from an error state is also an error state, so w' also reaches an abstract error state. Note that this w' is not necessarily feasible, only partially feasible for the variables of the precision. However, in such a case, if the abstract state space exploration is part of an abstraction-refinement scheme, the refinement step of the verification procedure will realize that w' is not feasible, and it will refine the abstraction.

5 Static Abstraction-Aware Partial Order Reduction Algorithm

In this section, we present a case study of our general approach by showing the soundness of an actual partial order reduction algorithm extended in the proposed abstraction-aware manner. Several algorithms have been presented in the literature for partial order reduction [1, 22, 24]. There are two main approaches: *static* and *dynamic* POR [7]. In the static version, the model (i.e., the CFA of the program) is analyzed, and the reduced state space is generated based on this static analysis. The dynamic approach constructs the reduced state space during model checking. Although abstraction-aware partial order reduction uses on-the-fly information about the current abstraction, its core POR algorithm can be implemented in both a static and a dynamic fashion.

Although our abstraction-aware extension can be applied to any partial order reduction algorithm, dynamic approaches tend to have much more complex formulations [1, 2]. Therefore, in this paper, we will restrict ourselves to the presentation of a static approach. Dynamic POR algorithms can be extended to use the abstraction-aware dependency relation in an analogous way; and we also include a dynamic POR algorithm in our experimental evaluation. Our static POR algorithm presented in this section is similar to existing static POR algorithms that calculate persistent or source sets [2, 24, 35]: we only add the abstraction-aware extension and prove the soundness of the extended algorithm. Before presenting the algorithm that calculates source sets, the following notions are introduced (similar concepts can be found in [2]):

Definition 5.1 (May-enabled action in a state) An action α is *may-enabled* in a state s, if $\alpha \in \mathit{enabled}(s)$ or α can become enabled after a sequence of transitions from processes other than p_α.

An action $\alpha \notin \mathit{enabled}(s)$ can be *may-enabled* in a state s when the source location of α is $s(p_\alpha)$, but the guard condition of α evaluates to false in s.[5]

Definition 5.2 (Future actions) For an $\alpha \in \mathit{enabled}(s)$, $\mathit{future_actions}(s, \alpha)$ is a set of actions: $\beta \in \mathit{future_actions}(s, \alpha)$ iff there is a transition sequence $w = w_1 ... w_n$ from s, where $w_n = \beta$, and the first action β_0 of p_β in w is either α or $\beta_0 \notin \mathit{enabled}(s)$.

The condition on β_0 in the definition of *future_actions* implies that β_0 is may-enabled in s. If β is in the same process as α, then $\beta_0 = \alpha$. Otherwise, β_0 is not enabled in s.

We can easily compute an over-approximation of $\mathit{future_actions}(s, \alpha)$ by analyzing the static model of the program. Initially, the actions of process p_α are collected with a graph traversal of the CFA of p_α. An action $\beta \notin \mathit{enabled}(s)$ from another process that is may-enabled in s can be enabled by an action γ reached in the CFA of p_α (when γ writes a variable that β uses in its guard condition). Then, $\mathit{future_actions}(s, \beta)$ is computed recursively to collect more future actions.

With the help of *future_actions*, we can compute source sets in a state. The current state s and the precision Π of the current abstraction are the inputs of Algorithm 1. Initially, any enabled action (or practically all enabled actions of a single process) is put in the source set(-to-be) P. As long as any new action is added to P, the following

[5] If processes can be created or terminated dynamically, actions can be may-enabled in other ways, too; e.g., first actions of processes and join operations.

Algorithm 1: Calculating a Source Set from s with Π

Input: s, Π
Output: S `/* Source set in s */`
1 $S \leftarrow \{\alpha\}$ for some $\alpha \in enabled(s)$
2 **while** *any new item is added to* S **do**
3 $\quad S \leftarrow S \cup \{\alpha \ : \ \alpha \in enabled(s) \setminus S, \ \exists \beta \in future_actions(s, \alpha), \exists \gamma \in S$ such that $(\beta, \gamma) \in D_\Pi\}$

is repeated: $future_actions(s, \alpha)$ is calculated for each $\alpha \in enabled(s) \setminus P$. If there is any action in $future_actions(s, \alpha)$ that is dependent with an action in P based on the abstract dependency relation D_Π, α is added to P.

The correctness of Algorithm 1 is stated by the following theorem:

Theorem 5.1 *A set S returned by Algorithm 1 for a state s and precision Π is a source set in s for $W_\Pi(s)$.*

Before proving theorem, we note the following property of software. Branches in a program are defined in an if-elseif-else manner, that is the execution can proceed on a branch independent of the variable assignment.

Property 5.1 For any location l of a CFA the disjunction of guard conditions of all outgoing actions from l is *true*.[6]

We also need the following lemma to prove Theorem 5.1. It states that a partially feasible trace w can be extended to a longer partially feasible trace with an action that is enabled in the first concrete state of a partial concretization of w and that is independent with all actions in w.

Lemma 5.1 *Let Π be the precision of the abstraction, $P = vars(\Pi)$, s be an abstract state, and $w = w_1 ... w_n$ be a partially feasible trace from s for P. Let $c_0, ..., c_n$ be a partial concretization of w from s, and let $\alpha \in enabled(c_0)$ such that $(\alpha, w_i) \notin D_\Pi$ for each $1 \leq i \leq n$. Then, $w.\alpha$ is also a partially feasible trace from s for P.*

Proof. Throughout the proof, $1 \leq i \leq n$ and $0 \leq j \leq n$. Let $W = \bigcup_i vars(w_i)$, that is, the variables used by any action of w; and let $c_\alpha = \alpha(c_0)$.

We define the concrete states d_j such that $d_j =^{vars(\alpha) \setminus W} c_\alpha$, and $d_j(v) = c_j(v)$ otherwise. As for the CFA locations, $d_j(p) = c_j(p)$ for each process $p \neq p_\alpha$, and $d_j(p_\alpha) = c_\alpha(p_\alpha)$.

It can be easily checked that the conditions of Lemma 3.1 are met by the concrete states $c_0, d_0, ..., d_n$. For c_0, there is c_α with $c_0 \xrightarrow{\alpha} c_\alpha$, and $c_\alpha =^P d_0$ (see the definitions of these states and Observation 1 for α and w_i). From d_0, the conditions are also met. Using the fact that $c_0, ..., c_n$ is a partial concretization of w, there are concrete states $c'_1, ..., c'_n$ conforming to the conditions in Definition 3.2. Let the concrete states $d'_0 = \alpha(c_0)$ and $d'_i = w_i(d_{i-1})$. Since $d_j =^W c_j$ by definition of these states, $d'_j =^W c'_j$ also holds as performing an action on states with the same value for each variable used by the action (W) yields the same resulting values. Also, from Definition 3.2, $c'_j =^P c_j$. Thus, $d'_j =^{W \cap P} d_j$. Applying Observation 2 to w_i, we can conclude that $d'_j =^P d_j$.

[6] An action without a guard is technically an action whose guard is *true*.

Naturally, $d'_j(p) = d_j(p)$ for each process p. Altogether, $c_0, d_0, ..., d_n$ and $d'_0, ..., d'_n$ indeed satisfy the conditions of Lemma 3.1.

Then, by Lemma 3.1, the trace $\alpha.w$ is partially feasible from s for P. Since $(\alpha, w_i) \notin D_\Pi$, $\alpha.w \approx^\Pi w.\alpha$ holds, which implies by Theorem 3.1 that $w.\alpha$ is also partially feasible from s for P. □

Proof of Theorem 5.1. Let us check the definition of source sets, that is, for each trace from s, one of its weak initials is in S. Let $w = w_1...w_n \in W_\Pi(s)$ be a partially feasible abstract trace from s. We have two cases:

1. $\exists w_i \in S$ for some $1 \leq i \leq n$ (that is w contains an action from S). Let $w_f \in S$ be the first occurrence of an element of S in w: $w_j \notin S$ for $1 \leq j < f$.
   Then, $(w_j, w_f) \notin D_\Pi$. To see this, assume the opposite: there is a w_d dependent with w_f based on D_Π, and $d < f$. Since w_f is the first action from S in w, w_d can be reached from s with actions from processes that do not have actions in S. That is, $w_d \in \mathit{future_actions}(s, \alpha)$ for some $\alpha \in \mathit{enabled}(s) \setminus S$. In this case, Algorithm 1 would have added α to S in line 3 (with w_d as β, w_f as γ, and α as α using the notation of the algorithm). This did not happen, so our indirect supposition is wrong.
   Since w_f is independent with all actions preceding w_f in w, $w_f.(w \setminus w_f) \approx^\Pi w$ which means by definition that $w_f \in WI_s^\Pi(w)$. So one of the weak initials of w is in S, indeed.
2. $\nexists w_i \in S$ for $1 \leq i \leq n$.
   This supposition implies that all actions in w are independent with all actions in S based on D_Π. Assume the opposite: there is a w_d for some $1 \leq d \leq n$, and $\gamma \in S$ such that $(w_d, \gamma) \in D_\Pi$. Reasoning is similar to case 1: since $\nexists w_i \in S$, $w_d \in \mathit{future_actions}(s, \alpha)$ for some $\alpha \in \mathit{enabled}(s) \setminus S$. Then, Algorithm 1 would have added α to S which did not happen.
   Since all actions in w are independent with all actions in S, there is at least one $\alpha \in S$ that is a weak initial of w. For this, take a partial concretization $c_0, ..., c_n$ of w from s ($c_0 \models s$). Take any action $\delta \in S$: if $\delta \in \mathit{enabled}(c_0)$, let $\alpha = \delta$. Otherwise, there is a $\delta' \in \mathit{enabled}(c_0)$ of process p_δ (starting from the same CFA location as δ) based on Property 5.1. Furthermore, $\delta' \in S$ since $(\delta', \delta) \in D_\Pi$ (as they belong to the same process) and $\delta' \in \mathit{future_actions}(s, \delta')$, so Algorithm 1 adds δ' to S. In this case, let $\alpha = \delta'$. Thus, $\alpha \in S$, which implies that α is independent with all actions in w. Since $\alpha \in \mathit{enabled}(c_0)$, and $(\alpha, w_i) \notin D_\Pi$, we can infer that $\alpha.w \approx^\Pi w.\alpha$, and $w.\alpha$ is partially feasible from s based on Lemma 5.1. This means by definition that $\alpha \in WI_s^\Pi(w)$, indeed. So one of the weak initials of w is in S, again.

□

6 Experimental Evaluation

In this section, we perform an experimental evaluation of the proposed algorithmic extensions presented in the paper. First, we introduce the experiment plan, the research questions we aim to answer, and the technical configuration details in Section 6.1. Then, we present and discuss the results of the experiment in Section 6.2 with respect to the research questions. Finally, we outline the conclusions of our experiments in Section 6.3, and discuss possible threats to their validity in Section 6.4.

6.1 Experiment Design

The goal of our experiment is to evaluate the effect of using abstraction-aware partial order reduction in the abstract state space and observe the theoretical and practical performance implications of using the proposed approach. Therefore, we implemented different base POR algorithms in multiple verification tools to obtain results that generalize well for further verification settings. Namely, we implemented the *static partial order reduction* algorithm presented in Section 5 (`SPOR`), and the *Source-DPOR* algorithm of *Abdulla et al.* (`DPOR`) [1]. We created an implementation of the static and dynamic version in the verification framework Theta[7] [42], and a proof-of-concept implementation of the static approach in `CPAchecker`[8] [11]. Each implementation can be used without and with the proposed abstraction-aware extension (the latter being denoted by an `AA` prefix added to the configuration name).

To demonstrate that the proposed extension can be applied in different abstract domains, we perform the practical evaluation using two abstract domains: the *explicit value* abstract domain (`EXPL`) and the *predicate* abstract domain (`PRED`) introduced in Section 2.2.

6.1.1 Research Questions

To evaluate the proposed approach, we aim to answer the following main research questions.

RQ1 Does the abstraction-aware extension enable greater state space reduction then using the traditional partial order reduction approach?
RQ2 What are the performance implications of using the proposed abstraction-aware extension of partial order reduction over a wide range of benchmark programs?
RQ3 How does the practical performance compare to the theoretically exponential gain over the program family introduced in Figure 1.2?

Regarding RQ1, we aim to confirm that the fewer variables the precision tracks, the greater reduction can be achieved using the abstraction-aware dependency relation compared to the syntactic dependency relation. We expect this based on the fact that the size of the abstraction-aware dependency relation decreases as the precision tracks less information. In RQ2, we want to investigate how the reduction of the abstract state space size with a given precision affects the overall performance of a complete verification algorithm following an abstraction-refinement iteration. For RQ1-RQ2, we are interested in the effect of the proposed extension while using different base partial order reduction algorithms (static and dynamic) and different abstract domains (explicit and predicate). For the overall performance implications in RQ2, we also investigate the verification procedures implemented in different frameworks (Theta and `CPAchecker`). RQ3 aims to confirm whether the theoretically expected exponential gain can be reproduced in a practical verification workflow using the proposed approach.

[7] Available at: https://github.com/ftsrg/theta/commit/c83ee7d
[8] Available at: https://gitlab.com/levente.bajczi/cpachecker/-/commit/40936e51

6.1.2 Experimental Configuration

We use the concurrency safety benchmark suite[9] of SV-COMP [14] containing 725 verification tasks[10] for RQ1-RQ2, and a direct implementation of Figure 1.2 for RQ3 with $N := 2^{0 \leq i \leq 8}$. The configurations were executed on virtual machines with AMD Ryzen 7 PRO 5750G processors, and 4 dedicated CPU cores were allocated to each task. Experiments regarding RQ1-RQ2 were executed with a 15 minute time limit, while we allowed the executions for RQ3 to run for 30 minutes.

For RQ1, we are interested in the sizes of the abstract state spaces. Therefore, we do not stop at the first reached abstract error state, rather explore the full reduced state space to have a sensible metric about the abstract state space size. Then, the refinement of the abstraction is done using an interpolant calculating SMT solver for all discovered abstract error traces, if necessary. Since this full exploration has a natural computational overhead, we use a more efficient strategy for the performance-focused research questions RQ2-RQ3 where we stop at the first reached abstract error state to check the feasibility of the discovered error trace and refine the abstraction right away, if needed. For RQ3, we only use the predicate abstract domain, which is expected to solve that task; for this task, we also use a configuration that initializes the precision with predicates automatically extracted from the branching conditions of the program.

Furthermore, we use a depth-first state space exploration strategy and thread-safe large-block encoding (one global operation per block). For the predicate abstract domain, we use *Cartesian* predicate abstraction and *atoms* as the basis of predicate splitting [27]. These configuration options, while designed with Theta [42] in mind, have been closely matched by the configuration we use for `CPAchecker` [11] with minor differences, mostly regarding solver choice and integer encoding. For the explicit domain, we use a maximum number of enumerated successor states of 1 guaranteed by using the abstract *unknown* value and the strongest post operator under abstract variable assignment to preserve action-determinism, which is required from the abstract state space (see Section 2). We do not execute `CPAchecker`with its value analysis as it is imprecise by design (accepts spurious counterexamples).

All tools and configurations used for the experimental evaluation of this paper are published as an artifact [40]: for more configuration details, we refer the interested reader to this artifact.

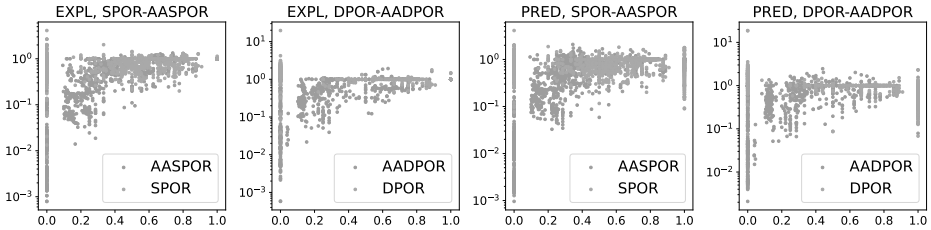
6.2 Experimental Results

We executed the different experiment configurations detailed in the previous section and obtained the following results:

RQ1 To get a full picture of the scale of reduction enabled by abstraction-aware POR compared to its traditional counterpart, we performed a full exploration of the abstract state space in each iteration of CEGAR and collected the size of the explored state space by iteration and task. The level of abstraction was also collected, which is characterized by the proportion of global variables tracked in the precision: the fewer variables are tracked, the coarser the abstraction, and the smaller the size of the abstraction-aware dependency relation. Each data point in the scatter plots of

[9] Available at: gitlab.com/sosy-lab/benchmarking/sv-benchmarks/-/commit/ca46fe67

[10] For reference: the parser of Theta fails for 123 of these tasks.



Q	N_{AA}	N_B	r	p
1	519	316	-0.62	0.00
2	298	214	-0.25	0.00
3	269	697	0.23	1.00
4	135	313	-0.10	0.02

Q	N_{AA}	N_B	r	p
1	422	246	-0.77	0.00
2	171	51	-0.34	0.00
3	186	83	-0.24	0.00
4	111	132	-0.03	1.00

Q	N_{AA}	N_B	r	p
1	530	319	-0.64	0.00
2	457	173	-0.16	0.00
3	244	317	0.12	1.00
4	249	320	-0.05	0.39

Q	N_{AA}	N_B	r	p
1	410	254	-0.79	0.00
2	244	58	-0.22	0.00
3	296	215	-0.40	0.00
4	199	218	-0.04	0.68

Fig. 6.1: Size of the explored abstract state space relative to the abstract state space size in the last iteration of the same verification procedure (y) with respect to the level of abstraction, i.e., the ratio of global variables tracked in the precision (x). The tables show the statistical significance testing results: Q is the quartile on the x-axis, N_B is the number of baseline data points, N_{AA} is the number of data points belonging to the abstraction-aware configuration, r is the effect size revealing the relative difference from the test statistic value expected under the null hypothesis, p is the *p-value* of the test. Quartiles with significant advantage of `AAPOR` are colored with green, red means no significant advantage of `AAPOR` compared to the baseline.

Figure 6.1 correspond to a CEGAR iteration for some program in the benchmark suite: the x-axis shows the proportion of global variables in the precision, while the y-axis shows the relative size of the state space compared to the state space size in the last iteration of CEGAR (which has the most tracked variables in the precision – typically tracking all variables in this benchmark set) for the same program. We only include data points for programs where both the baseline and the abstraction-aware configurations finished and returned the expected verdict within the time limit. We excluded the two programs from the data sets where all configurations (both baseline and `AAPOR`) returned false positive verdicts (infeasible counterexamples), as this feasibility checking bug is orthogonal to our contribution. There were no other incorrect verdicts.

It is easy to see that the `AAPOR` configurations indeed tend to explore a much smaller state space in iterations with a precision tracking fewer variables (left part of the plots), while the size of the explored state space with the baseline configurations is roughly the same in all configurations. This confirms our expectation that abstraction-aware POR can achieve significantly greater reduction when the precision tracks less information.

Note that there is some randomness in the POR algorithms (e.g., the local decision of choosing from the smallest source sets in the states may affect the number of explored states in the end). That is why it is possible to have y values greater than one in these plots. Also note that there are many iterations with predicate abstraction where the precision variable ratio (x value) is 1, while it is not so frequent with explicit value abstraction. This is because it is a frequent situation in predicate abstraction that we already have predicates containing all variables, but we still need extra predicates to prove the program safe. On the other hand,

tool	domain	por	average explored actions per task		total CPU time (s)		solved tasks	
Theta	EXPL	SPOR	16187		5214		337	
		AASPOR	13461	↓ -16.84%	5000	↓ -4.1%	347	↑ 2.97%
		DPOR	17072		20509		316	
		AADPOR	16074	↓ -5.85%	18122	↓ -11.64%	327	↑ 3.48%
	PRED	SPOR	12534		15515		372	
		AASPOR	11009	↓ -12.17%	15543	↑ 0.18%	373	↑ 0.27%
		DPOR	6102		16063		335	
		AADPOR	5819	↓ -4.64%	16866	↑ 5.0%	333	↓ -0.6%
CPAchecker	PRED	SPOR	48528		28381		241	
		AASPOR	35644	↓ -26.55%	27503	↓ -3.09%	241	↑ 0.0%

Table 6.1: Results of the evaluation for RQ2 on sv-benchmarks

tracking all variables explicitly means that it is practically a concrete state space exploration (apart from *unknown* values) ending with a definitive result regarding the existence of a feasible error trace, so there are no more refinement rounds in such cases.

To confirm that the visually convincing advantage of abstraction-aware POR is indeed statistically significant, we test our hypothesis with an unpaired non-parametric rank test:*Mann–Whitney U test*. We partition our data sets into quartiles based on equally sized intervals along the x-axis (proportion of tracked variables). We perform the hypothesis test in each quartile. The null hypothesis says that the data points belonging to the baseline and the abstraction-aware configurations come from the same distribution (they have the same median). The alternative hypothesis states that the abstraction-aware configuration yields a smaller relative state space size compared to its baseline. We use a significance level of 0.01: we reject the null hypothesis when the *p-value* of the test is below this threshold. We also apply *Bonferroni correction* to the p-values to reduce the probability of incorrectly rejecting the null hypothesis when it is true in any of the four quartiles. The results are reported in the tables in Figure 6.1 for each configuration pair. The tests confirm that the `AAPOR` configurations indeed have an advantage when the precision tracks fewer variables. However, this advantage vanishes as the precision tracks more and more variables (see the non-significant results in quartiles 3 or 4). These results are completely in line with our expectations.

RQ2 We list the overall verification metrics for our benchmark tests in Table 6.1. The first three columns contain the configurations, then we include the aggregated statistics for the number of explored actions and CPU time for the commonly solved tasks for each baseline and abstraction-aware configuration pair; finally we include the number of correctly solved problems in the benchmark suite by the individual configurations. Theta produced no incorrect verification verdicts, while there were three programs in case of `CPAchecker` where no configuration yielded a correct result: we excluded these programs.

The results reveal that the abstraction-aware configurations can reduce the aggregated number of explored transitions in each case, achieving greater reductions with the static POR algorithms. The total CPU time is also improved in some cases, but there is an increase in Theta when using the predicate abstract domain. A possible explanation for this is that using the abstraction-aware version of POR

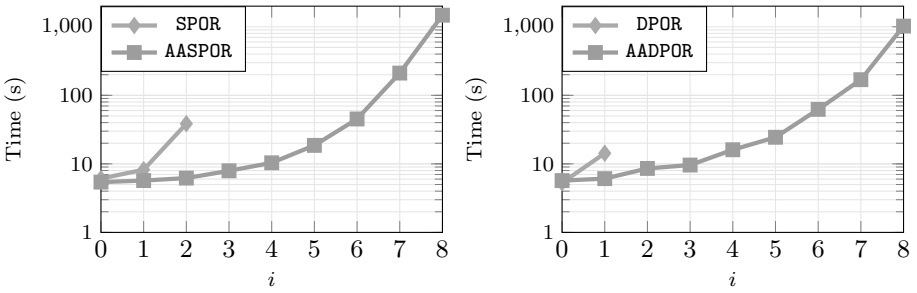


Fig. 6.2: Execution time for RQ3 given i for $N := 2^i$ in Figure 1.2.

can more frequently find infeasible abstract error traces leading to more refinement rounds. As the refinement step is computationally more demanding for predicate abstraction (interpolant calculation and new predicate generation), the increased number of refinements counterbalances the time spared by exploring fewer actions in the abstraction phase. The same overall effect can be observed in the number of solved tasks where a considerable increase only happens in case of the explicit value domain. The `CPAchecker` implementation confirms these findings: the number of explored actions is considerably decreased, however, the overall performance benefits are less pronounced.

While dynamic POR algorithms can generally achieve better reduction than static approaches [22], they are disadvantaged in an abstract state space exploration when covering is used [31]. Covering is applied during abstract state space exploration when an abstract state is reached that is over-approximated by another abstract state reached earlier during the exploration. The exploration stops at these states as all possible behavior is already explored from the other (more general) state. However, extra caution needs to be taken when using a DPOR algorithm and covering simultaneously so that all necessary thread interleavings are explored which poses considerable computational overhead [31]. Our experiments also confirm this situation as the `DPOR` configurations can solve fewer problems in the benchmark suite.

RQ3 The experiment results plotted in Figure 6.2 clearly reveal the great advantage of the abstraction-aware POR configurations over their traditional counterparts. While the `SPOR` and `DPOR` configurations using the syntactic dependency relation can solve verify these programs up to $N = 4$ within the time limit, the abstraction-aware `AASPOR` and `AADPOR` configurations could solve these tasks up to $N = 256$. Indeed, the proposed abstraction-aware POR algorithms scale much better on certain programs.

6.3 Evaluation Summary

As demonstrated by the data in Figure 6.2, there are certain scenarios where the abstraction-aware extension of POR may lead to vastly improved verification performance. The experiments also confirmed that the proposed abstraction-aware extension has a significant effect when the precision of the abstraction contains information about

only a small subset of global variables. We report mixed results regarding the overall verification performance on a large benchmark suite of concurrent programs. While the proposed approach consistently decrease the number of actions needed to verify a program, it only contributes a considerable improvement to the verification performance when using some abstract domains and partial order reduction algorithms. However, performance benefits vanish in some cases and are not observable in general for any abstract domain or POR method. We obtained similar results with the implementation of the proposed technique in a different verification framework, which supports the generalizability of our results across different model checker implementations.

6.4 Threats to Validity

The following factors may influence the validity of the experiments.

Internal Validity. The consistency and the accuracy of the experiments were ensured by using the BenchExec framework [13]. Memory consumption statistics may deviate between executions due to the managed nature of the Java virtual machine; therefore, such metrics are not used (both Theta and `CPAchecker` run on the JVM). CPU time and, therefore, the number of solved tasks may be influenced by external factors such as other processes or environmental temperature fluctuations, so minute differences are disregarded.

External Validity. The lack of certain patterns in the benchmark suite (i.e., where abstraction is indeed useful and necessary) can lead to the diminished advantage of the proposed abstraction-aware partial order reduction algorithm on these benchmark programs. However, the SV-COMP benchmark suite is the *de-facto* standard for academic verification tool development, and we could still demonstrate the advantages of the proposed approach on these programs.

Construct Validity. In order to corroborate that the right metrics are used in the evaluation, we considered both the user-facing and the backend-related interactions of the verification tool. The number of solved tasks, and the overall CPU time necessary for the solution both directly affect the user's ability to verify programs at hand, and the decrease in expanded transitions and the size of the state space will influence the number of solver invocations, reducing the load on the entire system. Therefore, these metrics accurately represent the expected outcomes of the executions.

7 Related Work

Partial order reduction has been a field of active research in the last decades [1, 7, 22, 24, 36, 43]. Early POR methods [7, 24, 43] approximated the conflicts between actions statically. Later, a depth-first search manner dynamic partial order reduction (DPOR) algorithm was introduced [22], where dependence between actions is decided dynamically during the state space exploration looking at the exploration stack. Source DPOR is a dynamic POR algorithm [1] whose extension, Optimal DPOR [1] is proven to be optimal in the number of explored interleavings. Many optimizations exist for determining dependency where information is retrieved from the state space search context: in these works, actions are considered dependent only in certain states under certain conditions [4, 5, 20, 34, 45], although these conditions are typically not related to information about the applied abstraction.

Several works use POR in the context of an abstraction-based verification algorithm [8,20,25,29,31,37,39,44]. However, most of them do not take advantage of using information about the current abstraction in order to increase the reducing effect of POR, whereas this is the key concept of our proposed approach.

CPAchecker is a program verification framework that supports several analysis techniques, including Counterexample-Guided Abstraction Refinement (CEGAR) and POR [8]. However, the existing POR algorithm in CPAchecker (i.e., not the new implementation that we added for the evaluation of this work) is relatively simple: only thread-local operations of different threads are considered independent (where an operation is global if it accesses a global memory location and thread-local otherwise). That is, the application of POR is orthogonal to CEGAR in CPAchecker.

In the works of *Su et al.* [39], *Kroening et al.* [31] and *Wachter et al.* [44], the specific abstraction-based verification algorithm Impact is combined with a POR algorithm. Although some of them use conditional dependency, their conditions are similar to the guarded independence relation described by *Wang et al.* [45], and they do not exploit information about the applied abstraction to reduce dependency. *Kroening et al.* [31] also discuss the necessary extensions for DPOR algorithms when the abstraction-based algorithm uses *covering*. Covering is applied when a reached abstract state is over-approximated by another abstract state reached earlier. The exploration stops at these states as all possible behavior is already explored from the other state. Combining this technique with the depth-first style DPOR algorithms needs extra consideration [31] that our DPOR algorithm implemented for the evaluation also had to deal with. *Hansen et al.* [29] use POR for the abstraction-based verification of timed automata. Although they use information about the current abstraction, they define relations such that one order of execution simulates the other one. Our case is more general: any execution order can be selected for exploration based on our approach, and none of the executions have to be the over-approximation of the other.

Several works realize that even the traditional dependency relation is not a valid dependency relation [16,37]. The combination of POR and abstraction in the work of *Cimatti et al.* [16] is specific to the Explicit Scheduler and Symbolic Threads (ESST) algorithm, while our approach is general. In our general setting, two syntactically independent actions may disable each other in the abstract state space (see Figure 3.2 in Example 3.3) whereas they implicitly assume in their proof that such situation cannot happen. *Hansen* [28] also considers the application of partial order reduction in non-deterministic abstract state spaces. However, the work focuses on the challenges posed by non-determinism, and not on the generalization of the commutativity relation.

A central element of our work builds on the idea of abstract commutativity relations investigated by *Farzan et al.* [21]. They realize that the commutativity relation can also be relaxed in an abstraction-based setting. However, their theoretical analysis focuses on the properties of abstract commutativity relations and their combinations, and they do not investigate how these relations can be embedded in arbitrary POR algorithms. Instead, they only discuss this relaxed commutativity in the context of their earlier work on using commutativity relations in a proof-checking algorithm based on sleep sets and persistent sets [20]. Although several notions are analogous between our work and theirs, there are two main differences: our proofs are for arbitrary POR algorithms based on the notion of source sets, which are more general than persistent sets [2], and the soundness results of their algorithm build on the guarantee of having a representative of *every* Mazurkiewicz-trace ending in an accepting state, therefore having a full proof for every one of them [21]. In contrast, we use abstraction-aware independence

in the context of an abstract state-space exploration algorithm that interprets the program statements even before reaching the error location, therefore pruning infeasible executions earlier. Because of this, we might lose traces and not have a representative of some Mazurkiewicz-trace, making the soundness proof more involved, as we needed to prove that if we have no representative of a Mazurkiewicz-trace in the reduced abstract state space, then all of them would be infeasible, so no necessary information was lost (cf. Theorem 4.1 and Theorem 3.1). We also believe that examining the analogous concepts (e.g., partial feasibility vs. infeasibility proof of abstracted traces with projection to the proof) from a different perspective provides additional insight into their workings.

Using counterexample-guided refinement for synthesizing context-sensitive commutativity relations and combining it with abstraction-refinement of the state space has also been recently proposed [19]. The novel part of that work is orthogonal to ours: the commutativity relation is iteratively made more general as the algorithm learns that some actions commute in a given context, and plugging the iteratively refined commutativity into trace abstraction is based on the earlier work discussed above [21]. Our abstraction-aware independence relation presented here is global, not context-sensitive, but investigating whether it can be replaced with a context-sensitive one in a sound way (which is not always the case [20]) is a promising future direction, in which case an iteratively refined relation could also be used.

A simple combined use of partial order reduction and trace abstraction has also been proposed [15], but it uses only concrete commutativity, and analyzes the reduced program with trace abstraction.

Sousa et al. propose a combination of abstract interpretation and unfoldings for concurrent program analysis [37, 38], a technique similar to partial order reduction. While their proposed algorithm uses the underlying ideas of DPOR to make exploring the configurations of an unfolding more efficient, the algorithm is still based on the unfolding data structure, while our algorithm uses standard state space exploration with POR. Another difference is that their algorithm employs abstract interpretation without any refinement, while we integrated abstraction-aware POR into a counterexample-guided abstraction refinement loop. A third difference is that although they prove the soundness of using either the independence relation of abstract transformers or the independence relation inherited from the concrete semantics, our abstraction-aware independence does not fall into either category.

Finally, in a previous work, the authors of the present paper have already discussed the implicit combination of partial order reduction and abstraction-based methods [41]. In that work, an abstract on-the-fly data flow analysis is used to perform a cone-of-influence reduction in concurrent programs. The purpose of that work differs, although the statement simplification can have a similar side effect regarding the dependency relation as in the present paper (cf. statement abstraction in the work of *Farzan et al.* [21]). On the other hand, a complicated data flow analysis is used there that can reduce the size of the dependency relation. That approach is both algorithmically more expensive to compute (non-constant in the size of the program, whereas the abstraction-aware dependency relation in this paper can be computed in constant time) and over-approximating (i.e., the abstraction-aware dependency relation used in this paper is smaller). Moreover, that work does not investigate the general correctness conditions regarding the POR algorithm when combined with abstraction. Thus, this previous paper of ours does not subsume our current work in any way, even though we find it interesting to mention the connections.

References

1. Abdulla, P.A., Aronis, S., Jonsson, B., Sagonas, K.: Optimal dynamic partial order reduction. pp. 373–384. ACM (2014). https://doi.org/10.1145/2535838.2535845
2. Abdulla, P.A., Aronis, S., Jonsson, B., Sagonas, K.: Comparing Source Sets and Persistent Sets for Partial Order Reduction. Lecture Notes in Computer Science, vol. 10460, pp. 516–536. Springer (2017). https://doi.org/10.1007/978-3-319-63121-9_26
3. Abdulla, P.A., Aronis, S., Jonsson, B., Sagonas, K.: Source Sets: A Foundation for Optimal Dynamic Partial Order Reduction. J. ACM **64**(4), 25:1–25:49 (2017). https://doi.org/10.1145/3073408
4. Albert, E., Arenas, P., de la Banda, M.G., Gómez-Zamalloa, M., Stuckey, P.J.: Context-Sensitive Dynamic Partial Order Reduction. Lecture Notes in Computer Science, vol. 10426, pp. 526–543. Springer (2017). https://doi.org/10.1007/978-3-319-63387-9_26
5. Aronis, S., Jonsson, B., Lång, M., Sagonas, K.: Optimal Dynamic Partial Order Reduction with Observers. Lecture Notes in Computer Science, vol. 10806, pp. 229–248. Springer (2018). https://doi.org/10.1007/978-3-319-89963-3_14
6. Baelde, D., Delaune, S., Hirschi, L.: Partial Order Reduction for Security Protocols. CoRR **abs/1504.04768** (2015)
7. Baier, C., Katoen, J.: Principles of Model Checking. MIT Press (2008)
8. Beyer, D., Friedberger, K.: A Light-Weight Approach for Verifying Multi-Threaded Programs with CPAchecker. EPTCS, vol. 233, pp. 61–71 (2016). https://doi.org/10.4204/EPTCS.233.6
9. Beyer, D., Henzinger, T.A., Jhala, R., Majumdar, R.: The software model checker Blast. Int. J. Softw. Tools Technol. Transf. **9**(5-6), 505–525 (2007). https://doi.org/10.1007/s10009-007-0044-z
10. Beyer, D., Henzinger, T.A., Théoduloz, G.: Configurable Software Verification: Concretizing the Convergence of Model Checking and Program Analysis. Lecture Notes in Computer Science, vol. 4590, pp. 504–518. Springer (2007). https://doi.org/10.1007/978-3-540-73368-3_51
11. Beyer, D., Keremoglu, M.E.: CPAchecker: A Tool for Configurable Software Verification. Lecture Notes in Computer Science, vol. 6806, pp. 184–190. Springer (2011). https://doi.org/10.1007/978-3-642-22110-1_16
12. Beyer, D., Löwe, S.: Explicit-Value Analysis Based on CEGAR and Interpolation. CoRR **abs/1212.6542** (2012)
13. Beyer, D., Löwe, S., Wendler, P.: Reliable benchmarking: requirements and solutions. Int. J. Softw. Tools Technol. Transf. **21**(1), 1–29 (2019). https://doi.org/10.1007/s10009-017-0469-y
14. Beyer, D., Strejcek, J.: Improvements in software verification and witness validation: SV-COMP 2025. In: Gurfinkel, A., Heule, M. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 31st International Conference, TACAS 2025, Held as Part of the International Joint Conferences on Theory and Practice of Software, ETAPS 2025, Hamilton, ON, Canada, May 3-8, 2025, Proceedings, Part III. pp. 151–186. Lecture Notes in Computer Science, Springer (2025). https://doi.org/10.1007/978-3-031-90660-2_9
15. Cassez, F., Ziegler, F.: Verification of concurrent programs using trace abstraction refinement. In: Davis, M., Fehnker, A., McIver, A., Voronkov, A. (eds.) Logic for Programming, Artificial Intelligence, and Reasoning - 20th International Conference, LPAR-20 2015, Suva, Fiji, November 24-28, 2015, Proceedings. pp. 233–248. Lecture Notes in Computer Science, Springer (2015). https://doi.org/10.1007/978-3-662-48899-7_17
16. Cimatti, A., Narasamdya, I., Roveri, M.: Software Model Checking with Explicit Scheduler and Symbolic Threads. Log. Methods Comput. Sci. **8**(2) (2012). https://doi.org/10.2168/LMCS-8(2:18)2012
17. Clarke, E.M., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement for symbolic model checking. J. ACM **50**(5), 752–794 (2003). https://doi.org/10.1145/876638.876643
18. Clarke, E.M., Klieber, W., Novácek, M., Zuliani, P.: Model Checking and the State Explosion Problem. Lecture Notes in Computer Science, vol. 7682, pp. 1–30. Springer (2011). https://doi.org/10.1007/978-3-642-35746-6_1
19. Ebbinghaus, M., Klumpp, D., Podelski, A.: Counterexample-guided commutativity. In: Piskac, R., Rakamaric, Z. (eds.) Computer Aided Verification - 37th International Conference, CAV 2025, Zagreb, Croatia, July 23-25, 2025, Proceedings, Part III. pp. 347–369. Lecture Notes in Computer Science, Springer (2025). https://doi.org/10.1007/978-3-031-98682-6_18

20. Farzan, A., Klumpp, D., Podelski, A.: Sound sequentialization for concurrent program verification. In: Jhala, R., Dillig, I. (eds.) PLDI '22: 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation, San Diego, CA, USA, June 13 - 17, 2022. pp. 506–521. ACM (2022). https://doi.org/10.1145/3519939.3523727
21. Farzan, A., Klumpp, D., Podelski, A.: Stratified Commutativity in Verification Algorithms for Concurrent Programs. Proc. ACM Program. Lang. **7**(POPL), 1426–1453 (2023). https://doi.org/10.1145/3571242
22. Flanagan, C., Godefroid, P.: Dynamic partial-order reduction for model checking software. pp. 110–121. ACM (2005). https://doi.org/10.1145/1040305.1040315
23. Flanagan, C., Qadeer, S.: Predicate abstraction for software verification. pp. 191–202. ACM (2002). https://doi.org/10.1145/503272.503291
24. Godefroid, P.: Partial-Order Methods for the Verification of Concurrent Systems - An Approach to the State-Explosion Problem, Lecture Notes in Computer Science, vol. 1032. Springer (1996). https://doi.org/10.1007/3-540-60761-7
25. Govind, R., Herbreteau, F., Srivathsan, B., Walukiewicz, I.: Abstractions for the local-time semantics of timed automata: a foundation for partial-order methods. pp. 24:1–24:14. ACM (2022). https://doi.org/10.1145/3531130.3533343
26. Grumberg, O., Clarke, E.M., Peled, D.A.: Model checking (1999)
27. Hajdu, Á., Micskei, Z.: Efficient Strategies for CEGAR-based Model Checking. Journal of Automated Reasoning **64**(6), 1051–1091 (2020). https://doi.org/10.1007/s10817-019-09535-x
28. Hansen, H.: Abstractions for transition systems with applications to stubborn sets. Lecture Notes in Computer Science, vol. 10160, pp. 104–123. Springer (2017). https://doi.org/10.1007/978-3-319-51046-0_6
29. Hansen, H., Lin, S., Liu, Y., Nguyen, T.K., Sun, J.: Diamonds Are a Girl's Best Friend: Partial Order Reduction for Timed Automata with Abstractions. Lecture Notes in Computer Science, vol. 8559, pp. 391–406. Springer (2014). https://doi.org/10.1007/978-3-319-08867-9_26
30. Klumpp, D., Dietsch, D., Heizmann, M., Schüssele, F., Ebbinghaus, M., Farzan, A., Podelski, A.: Ultimate gemcutter and the axes of generalization - (competition contribution). Lecture Notes in Computer Science, vol. 13244, pp. 479–483. Springer (2022). https://doi.org/10.1007/978-3-030-99527-0_35
31. Kroening, D., Sharma, S., Wachter, B.: AbPress: Flexing Partial-Order Reduction and Abstraction. CoRR **abs/1410.6044** (2014)
32. de León, H.P., Haas, T., Meyer, R.: Dartagnan: Leveraging compiler optimizations and the price of precision (competition contribution). Lecture Notes in Computer Science, vol. 12652, pp. 428–432. Springer (2021). https://doi.org/10.1007/978-3-030-72013-1_26
33. Mazurkiewicz, A.W.: Trace Theory. Lecture Notes in Computer Science, vol. 255, pp. 279–324. Springer (1986). https://doi.org/10.1007/3-540-17906-2_30
34. Meijer, J., Kant, G., Blom, S., van de Pol, J.: Read, Write and Copy Dependencies for Symbolic Model Checking. Lecture Notes in Computer Science, vol. 8855, pp. 204–219. Springer (2014). https://doi.org/10.1007/978-3-319-13338-6_16
35. Overman, W.T., Crocker, S.D.: Verification of Concurrent Systems: Function and Timing. pp. 401–409. North-Holland (1982)
36. Peled, D.A.: Ten Years of Partial Order Reduction. Lecture Notes in Computer Science, vol. 1427, pp. 17–28. Springer (1998). https://doi.org/10.1007/BFb0028727
37. Sousa, M.: Abstractions and independence. Ph.D. thesis, University of Oxford, UK (2018), https://ethos.bl.uk/OrderDetails.do?uin=uk.bl.ethos.780457
38. Sousa, M., Rodríguez, C., D'Silva, V.V., Kroening, D.: Abstract interpretation with unfoldings. In: Majumdar, R., Kuncak, V. (eds.) Computer Aided Verification - 29th International Conference, CAV 2017, Heidelberg, Germany, July 24-28, 2017, Proceedings, Part II. pp. 197–216. Lecture Notes in Computer Science, Springer (2017). https://doi.org/10.1007/978-3-319-63390-9_11
39. Su, J., Tian, C., Yang, Z., Yang, J., Yu, B., Duan, Z.: Prioritized Constraint-Aided Dynamic Partial-Order Reduction. pp. 78:1–78:13. ACM (2022). https://doi.org/10.1145/3551349.3561159
40. Telbisz, C., Bajczi, L., Szekeres, D., Vörös, A.: Partial Order Reduction for Abstraction-based Verification of Concurrent Software - Artifact (Apr 2026), https://doi.org/10.5281/zenodo.19460508
41. Telbisz, C.F., Bajczi, L., Szekeres, D., Vörös, A.: On-the-fly cone-of-influence reduction for model checking concurrent software. vol. 15945 (2025). https://doi.org/10.1007/978-3-032-06847-7_9

42. Tóth, T., Hajdu, A., Vörös, A., Micskei, Z., Majzik, I.: Theta: a Framework for Abstraction Refinement-Based Model Checking. pp. 176–179 (2017). https://doi.org/10.23919/FMCAD.2017.8102257
43. Valmari, A.: Stubborn sets for reduced state space generation. Lecture Notes in Computer Science, vol. 483, pp. 491–515. Springer (1989). https://doi.org/10.1007/3-540-53863-1_36
44. Wachter, B., Kroening, D., Ouaknine, J.: Verifying multi-threaded software with impact. pp. 210–217. IEEE (2013)
45. Wang, C., Yang, Z., Kahlon, V., Gupta, A.: Peephole Partial Order Reduction. Lecture Notes in Computer Science, vol. 4963, pp. 382–396. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_29